Java Kodları – Satır Satır En Basit Anlatım

İçindekiler

- Nasıl okunur?
- Önce bunları bil: en temel parçalar
- PDF 1 · Variable ve tipler (1-10)
- PDF 1 · Operator'ler (11-19)
- PDF 2 · if ve switch (20-34)
- PDF 2 · for ve for-each (35-43)
- PDF 3 · while ve do-while (44-53)
- PDF 3 · break, continue, comments (54-65)
- PDF 4 · Class ve object (66-76)
- PDF 4 · Method ve constructor (77-88)
- PDF 5 · static ve this (89-99)
- PDF 5 · Overloading, overriding, super (100-113)
- PDF 6 · Inheritance türleri ve interface (114-124)
- PDF 6 · Casting, final, aggregation (125-129)

Nasıl okunur?

Bu not 6 PDF'teki **kodların hepsini** tek tek, en basit dille açıklar. Her kodda:

- Kodun içinde **her satırın yanında** ne yaptığı yazıyor (// ile başlayan kısım).
- Altında **Çıktı**: ekrana ne yazdığı.
- Altında **Benzetme**: günlük hayattan karşılığı.

Önce bunları bil: en temel parçalar

Her Java kodunda bunlar var. Bunları bilmeden kodu okuyamazsın.

```
public class Main {                              // "Main" adında bir kutu (class) aç
    public static void main(String[] args) {     // Program BURADAN başlar. Ezberle, her programda var
        System.out.println("Merhaba");           // Ekrana Merhaba yaz ve alt satıra geç
    }                                            // main bitti
}                                                // class bitti
```

Parça	En basit anlamı
class	Kutu. Kodlar kutunun içine yazılır
main	Programın **başlama düğmesi**. Java ilk buraya bakar
{ }	Bir bloğun başı ve sonu. Açtığın her { kapanmalı }
;	Cümle sonundaki nokta gibi. Her komut ; ile biter
System.out.println(x)	Ekrana x'i yaz, **sonra alt satıra geç**
System.out.print(x)	Ekrana x'i yaz, **aynı satırda kal**
System.out.printf("%4d", x)	Biçimli yaz: sayıyı 4 karakterlik yere sağa yasla
// yazı	Not. Java bunu okumaz, sadece insan için
"metin"	Çift tırnak = yazı (String)
'A'	Tek tırnak = tek harf (char)
=	**Ata** (sağdakini soldakine koy). Eşit mi demek DEĞİL
==	Eşit mi? diye **sor** (cevap true/false)
new	Yeni bir şey (object) üret
. (nokta)	"...'nın": dog.bark() = köpeğin havlaması
+ yazıyla	Yazıları yapıştırır: "Ali" + " Veli" → Ali Veli

Benzetme: class bir **ev planı**, main evin **giriş kapısı**, ; her cümlenin **noktası**, { } bir odanın **kapıları**.

PDF 1 · Variable ve tipler

Variable = içine değer koyduğun **etiketli kutu**. Kutunun tipi, içine ne koyabileceğini söyler: int kutusuna tam sayı, String kutusuna yazı.

1. Variable tanımlama

```
int age;                    // "age" adında tam sayı kutusu aç (içi şimdilik boş)
age = 21;                   // kutuya 21 koy
int year = 2026;            // kutuyu aç ve hemen içine 2026 koy
int a = 5, b = 10;          // tek satırda iki kutu: a=5, b=10
final double PI = 3.14159;  // final = kilitli kutu. Bir daha değiştirilemez
var city = "Gümüşhane";     // var = "tipi sen anla". Java içine yazı konduğunu görüp String yapar
age = age + 1;              // age'in içindekine 1 ekle: 21 → 22
System.out.println(age + " " + year);  // 22 2026
System.out.println(a + b);             // 15 (iki sayı toplanır)
System.out.println(PI + " " + city);   // 3.14159 Gümüşhane
// PI = 3.14;  → HATA: kilitli kutu değiştirilemez
```

Benzetme: int age; = boş bir kavanoz alıp üstüne "age" yazmak. age = 21; = kavanoza 21 koymak. final = kavanozun kapağını **yapıştırmak**.

2. Local variable ve scope (görünürlük alanı)

```
int num = 10;                           // main içinde kutu → main içinde her yerde görünür
System.out.println("num: " + num);      // num: 10
for (int i = 0; i < 2; i++) {           // i sadece bu for'un içinde yaşar
    int square = i * i;                 // square da sadece bu { } içinde yaşar
    System.out.println("square: " + square);  // square: 0, sonra square: 1
}
// System.out.println(square);          // HATA: square for bitince öldü
int total;                              // kutu açıldı ama içi boş
// System.out.println(total);           // HATA: boş kutuyu okuyamazsın
total = 5;                              // şimdi içine 5 koyduk
System.out.println("total: " + total);  // total: 5
```

Benzetme: Scope = **oda**. Mutfakta (for içinde) bıraktığın tabak, mutfaktan çıkınca yok olur. Local kutuyu doldurmadan okursan Java kızar.

3. Instance variable (her object'in kendi kutusu)

```
class Student {                         // Öğrenci kalıbı
    String name;                        // her öğrencinin kendi adı
    int age = 19;                       // her öğrencinin kendi yaşı, başlangıç 19
    Student(String name) {              // öğrenci oluşturulurken çalışır
        this.name = name;               // gelen adı bu öğrencinin name kutusuna koy
    }
}
Student s1 = new Student("Ali");        // 1. öğrenci: Ali
Student s2 = new Student("Ayşe");       // 2. öğrenci: Ayşe
s2.age = 21;                            // sadece Ayşe'nin yaşı 21 oldu
System.out.println(s1.name + " - " + s1.age);  // Ali - 19
System.out.println(s2.name + " - " + s2.age);  // Ayşe - 21
```

Benzetme: Her öğrencinin **kendi çantası** var. Ayşe'nin çantasına bir şey koymak Ali'nin çantasını değiştirmez.

4. Static variable (herkesin ortak kutusu)

```
class Student {
    static int count = 0;               // static = TEK kutu, tüm öğrenciler paylaşır
    String name;
    Student(String name) {
        this.name = name;
        count++;                        // her yeni öğrencide ortak sayaç 1 artar
    }
}
Student s1 = new Student("Ali");        // count = 1
Student s2 = new Student("Ayşe");       // count = 2
Student s3 = new Student("Mehmet");     // count = 3
System.out.println("Toplam: " + Student.count);  // Toplam: 3
s1.count = 99;                          // ortak kutuyu 99 yaptık (böyle yazmak önerilmez)
System.out.println(s2.count + " " + s3.count);  // 99 99 → herkes aynı kutuya bakıyor
```

Benzetme: Static = sınıftaki **ortak tahta**. Biri tahtaya 99 yazarsa herkes 99 görür.

5. Tam sayı tipleri

```
byte b = 100;                          // küçük kutu (-128 ile 127 arası)
short s = 30_000;                      // _ sadece okumayı kolaylaştırır, 30000 demek
int population = 85_000_000;           // normal tam sayı kutusu
long distance = 15_000_000_000L;       // ÇOK büyük sayı → long + sonuna L
int hex = 0xFF;                        // 0x = 16'lık sistem → 255
int bin = 0b1010;                      // 0b = ikilik sistem → 10
int max = Integer.MAX_VALUE;           // int'e sığan en büyük sayı: 2147483647
System.out.println(max + 1);           // -2147483648 → taştı, en başa döndü (overflow)
System.out.println(Long.MAX_VALUE);    // 9223372036854775807
```

Benzetme: Overflow = **eski araba kilometre sayacı**: 999999'dan sonra 000000'a döner. Java hata vermez, sessizce başa sarar.

6. Ondalıklı sayılar

```
float f = 234.5f;                  // float kullanırsan sonuna f ŞART
double d = 12.3;                   // ondalıklı sayının varsayılanı double
double sci = 1.5e3;                // 1.5 x 1000 = 1500.0
System.out.println(10 / 4);        // 2   → iki tam sayı bölünürse virgülden sonrası ATILIR
System.out.println(10 / 4.0);      // 2.5 → biri ondalıklıysa sonuç da ondalıklı
System.out.println(1.0f / 3);      // 0.33333334 (float daha az hassas)
System.out.println(1.0 / 3);       // 0.3333333333333333 (double daha hassas)
System.out.println(0.1 + 0.2);     // 0.30000000000000004 → bilgisayar 0.1'i tam tutamaz
System.out.println(0.1 + 0.2 == 0.3);  // false → ondalıkları == ile karşılaştırma
```

Benzetme: 10 / 4 = 2 → bölme yapıp **küsuratı çöpe atmak**. Pastayı 4 kişiye bölünce kalan dilimleri kimseye vermemek.

7. char ve boolean

```
char c = 'A';                       // tek harf, tek tırnakla
char u = '\u0041';                  // harfin kod numarasıyla yazılmış hâli → yine 'A'
char tr = 'ş';                      // Türkçe harf de olur
System.out.println(c + " " + u + " " + tr);   // A A ş
System.out.println(c + 1);          // 66 → harfle matematik yapınca harfin NUMARASI kullanılır (A=65)
System.out.println((char) (c + 1)); // B → numarayı tekrar harfe çevirdik
c++;                                // harfi bir sonrakine kaydır: A → B
System.out.println(c);              // B
System.out.println((int) 'a');      // 97 → 'a' harfinin numarası
boolean isPassed = 75 >= 50;        // 75, 50'den büyük mü? → true
boolean isEmpty = false;            // boolean sadece true veya false olur
System.out.println(isPassed + " " + !isEmpty);  // true true (! = tersini al)
```

Benzetme: Her harfin bir **sıra numarası** var (A = 65, a = 97). boolean bir **ışık düğmesi**: ya açık (true) ya kapalı (false).

8. Type casting (tip dönüştürme)

```
int a = 10;
double d = a;                          // küçük kutudan büyük kutuya: otomatik → 10.0
double price = 10.99;
int p = (int) price;                   // büyükten küçüğe: (int) yazmak ZORUNLU → 10 (küsurat gitti)
System.out.println(Math.round(price)); // 11 → yuvarlama bu
int big = 130;
byte b = (byte) big;                   // 130 byte'a sığmaz → -126 (taştı)
byte x = 10, y = 20;
// byte z = x + y;                     // HATA: byte + byte sonucu int olur
byte z = (byte) (x + y);               // zorla byte yap → 30
```

Benzetme: Küçük bardaktaki suyu büyük sürahiye dökmek (widening) sorunsuz. Sürahiyi küçük bardağa dökmek (narrowing) için **"taşarsa taşsın"** demen gerekir: (int), (byte).

9. Reference tipler (String, array, enum)

```
enum Color { RED, GREEN, BLUE }        // enum = sabit seçenek listesi
String str = "Merhaba";                // yazı
int[] arr = {1, 2, 3, 4, 5};           // 5 kutuluk sayı dizisi
Color clr = Color.BLUE;                // listeden BLUE seçildi
System.out.println(str.length());      // 7 → yazının harf sayısı (parantezli!)
System.out.println(str.toUpperCase()); // MERHABA → büyük harfli YENİ yazı (str değişmez)
System.out.println("Boyut: " + arr.length);  // Boyut: 5 → dizinin boyu (parantezsiz!)
System.out.println(clr);               // BLUE
String s2 = null;                      // null = kutu var ama içi boş, hiçbir şeyi göstermiyor
System.out.println(s2 == null);        // true
```

Benzetme: Reference = **adres kâğıdı**. Kutuda evin kendisi değil, evin adresi yazar. null = adres kâğıdı boş.

10. Wrapper class'lar

```
Integer x = 42;                          // int'i kutuya sar (autoboxing)
int y = x;                               // kutudan çıkar (unboxing)
int n = Integer.parseInt("123");         // "123" yazısını 123 sayısına çevir
double r = Double.parseDouble("2.5");    // "2.5" yazısını 2.5 sayısına çevir
String s = String.valueOf(3.5);          // 3.5 sayısını "3.5" yazısına çevir
System.out.println(n + 1);               // 124 → sayı olduğu için toplandı
System.out.println(r * 2);               // 5.0
System.out.println(s + 1);               // 3.51 → yazı olduğu için 1 yanına yapıştı!
System.out.println(Integer.MAX_VALUE);   // 2147483647
System.out.println(Character.isDigit('7'));     // true → '7' bir rakam mı? evet
System.out.println(Integer.toBinaryString(y));  // 101010 → 42'nin ikilik hâli
```

Benzetme: Wrapper = sayıyı **hediye paketine** koymak. ArrayList gibi yapılar çıplak sayı kabul etmez, paketli ister. parseInt = **yazıyı sayıya çeviren tercüman**.

PDF 1 · Operator'ler

Operator = sayılar/değerler üzerinde işlem yapan işaret: +, -, ==, && gibi.

11. Aritmetik (dört işlem + kalan)

```
int a = 10, b = 3;
System.out.println(a + b);     // 13 → topla
System.out.println(a - b);     // 7  → çıkar
System.out.println(a * b);     // 30 → çarp
System.out.println(a / b);     // 3  → böl, küsuratı AT (iki tam sayı)
System.out.println(a % b);     // 1  → bölümden KALAN (10 = 3x3 + 1)
System.out.println(-10 % 3);   // -1 → kalanın işareti soldaki sayıdan gelir
System.out.println(10.0 / 3);  // 3.3333333333333335 → ondalıklı bölme
System.out.println(7.5 % 2);   // 1.5 → ondalıkta da kalan alınır
System.out.println(10.0 / 0);  // Infinity → ondalıkta sıfıra bölme = sonsuz
// System.out.println(10 / 0); → ÇÖKER: ArithmeticException (tam sayı sıfıra bölünmez)
```

Benzetme: % = **paylaştırmadan artan**. 10 elmayı 3 kişiye eşit dağıtırsan her biri 3 alır, **1 elma artar** → 10 % 3 = 1. Çift mi tek mi bakmak için: n % 2 == 0 ise çift.

12. ++ ve -- (bir artır / bir azalt)

```
int x = 10;
System.out.println(+x);    // 10
System.out.println(-x);    // -10 → işaretini çevir
System.out.println(x++);   // 10 → ÖNCE yaz (10), SONRA artır (x artık 11)
System.out.println(x);     // 11
System.out.println(++x);   // 12 → ÖNCE artır (12), SONRA yaz
System.out.println(x--);   // 12 → önce yaz, sonra azalt (x artık 11)
System.out.println(--x);   // 10 → önce azalt, sonra yaz
boolean ok = true;
System.out.println(!ok);   // false → ! = tersini al
System.out.println(~5);    // -6 → bitleri ters çevir (kural: ~x = -x - 1)
```

Benzetme: x++ = **"önce fotoğrafını çek, sonra büyüt"**. ++x = **"önce büyüt, sonra fotoğrafını çek"**. Sıra, ++'nın nerede durduğuna bakar.

13. Karşılaştırma ve mantık (&&, ||, !)

```
int a = 10, b = 20, c = 5;
System.out.println(a < b);             // true  → 10, 20'den küçük mü?
System.out.println(a == b);            // false → eşit mi?
System.out.println(a != b);            // true  → eşit DEĞİL mi?
System.out.println(a < b && b > c);    // true  → && = VE: ikisi de doğru olmalı
System.out.println(a > b || a > c);    // true  → || = VEYA: biri doğru yeter
System.out.println(!(a < b));          // false → doğrunun tersi
if (a < c && ++a > 0) { }              // a < c yanlış → && ikinci kısma BAKMAZ, ++a çalışmaz
System.out.println("a = " + a);        // a = 10
if (a < c & ++a > 0) { }               // tek & ikisine de bakar → ++a çalışır
System.out.println("a = " + a);        // a = 11
```

Benzetme: && = **"hem kimlik hem bilet"**: kimlik yoksa bilete bakmaya gerek yok, içeri giremezsin (short-circuit). || = **"ya nakit ya kart"**: nakit varsa karta bakmazsın.

14. Bitwise ve shift (bit kaydırma)

```
System.out.println(10 << 2);    // 40 → sola 2 kaydır = 10 x 2 x 2
System.out.println(15 << 4);    // 240 → 15 x 16
System.out.println(20 >> 2);    // 5  → sağa 2 kaydır = 20 / 4
System.out.println(20 >> 3);    // 2  → 20 / 8 = 2.5 → aşağı yuvarla
System.out.println(-20 >> 2);   // -5 → işaret korunur
System.out.println(-20 >>> 2);  // 1073741819 → >>> işareti umursamaz, dev bir pozitif çıkar
System.out.println(Integer.toBinaryString(20));       // 10100 → 20'nin ikilik yazımı
System.out.println(Integer.toBinaryString(20 >> 2));  // 101   → sağdan 2 hane silindi
```

İşlem	Sonuç	Basit kural
10 | 5	15	| (VEYA): iki sayının bitlerinden biri 1 ise 1
11 & 5	1	& (VE): ikisi de 1 ise 1
11 ^ 5	14	^ (XOR): farklıysa 1

Benzetme: << = sayının **sonuna sıfır eklemek** gibi (onluk sistemde 5 → 50 gibi, ama burada 2 katı). >> = **sondan hane silmek** (yarıya bölmek).

15. Atama kısaltmaları (+=, -=…)

```
int a = 10;
a += 3;   // a = a + 3 → 13
a -= 4;   // a = a - 4 → 9
a *= 2;   // a = a * 2 → 18
a /= 4;   // a = a / 4 → 4
a %= 3;   // a = a % 3 → 1
short s = 10;
s += 5;                 // çalışır: += gizlice tipi düzeltir
// s = s + 5;          // HATA: s + 5 int olur, short'a sığmaz
s = (short) (s + 5);   // elle düzeltince çalışır → 20
int x, y, z;
x = y = z = 7;         // sağdan sola: z=7, y=z, x=y → hepsi 7
System.out.println(x + y + z);  // 21
```

Benzetme: a += 3 = **"cebindekine 3 lira ekle"**. Uzun yazmak yerine kısa yazma.

16. Ternary (tek satırlık if)

```
int a = 2, b = 5;
int min = (a < b) ? a : b;              // a < b mi? evet → a'yı al → min = 2
int x = 10, y = 15, z = 13;
int max = (x > y) ? (x > z ? x : z)     // x > y mi? hayır → alt satıra git
                  : (y > z ? y : z);    // y > z mi? evet → y → max = 15
int number = 13;
String type = (number % 2 == 0) ? "çift" : "tek";   // 13 % 2 = 1 → "tek"
int score = 72;
System.out.println(score >= 50 ? "Geçti" : "Kaldı");  // Geçti
```

Benzetme: koşul ? evetse : hayırsa → "Yağmur var mı? **Evet**se şemsiye : **hayır**sa güneş gözlüğü."

17. İşlem önceliği

```
10*10/5+3-1*4/2
// 1) Önce * ve / (soldan sağa): 10*10 = 100, 100/5 = 20, 1*4 = 4, 4/2 = 2
// 2) Sonra + ve -:  20 + 3 - 2 = 21
```

Benzetme: Okuldaki **çarpma-bölme önce, toplama-çıkarma sonra** kuralının aynısı. Emin değilsen **parantez** kullan.

18. İlk bakış: for + if + while birlikte

```java
int sum = 0;                                    // toplamı tutacak kutu
for (int j = 1; j <= 10; j++) {                 // j: 1'den 10'a kadar say
    sum = sum + j;                              // her sayıyı toplama ekle
}
System.out.println("1..10 toplamı: " + sum);    // 55
if (sum > 50) {                                 // toplam 50'den büyük mü?
    System.out.println("Toplam 50'den büyük"); // evet → bu yazılır
} else {
    System.out.println("Toplam 50 veya daha küçük");
}
int i = 0;
while (i <= 10) {                               // i 10'u geçmediği sürece dön
    System.out.print(i + " ");                  // 0 2 4 6 8 10
    i += 2;                                     // 2'şer artır
}
```

19. String karşılaştırma: == mi, equals mı?

```java
String a = "Java";
String b = new String("Java");            // new → ayrı, yepyeni bir yazı nesnesi
String c = "Java";                        // aynı yazı havuzdan geldi
String d = "JAVA";
System.out.println(a == b);               // false → == "aynı kutu mu?" sorar. Değil
System.out.println(a == c);               // true  → tesadüfen aynı kutu (havuz)
System.out.println(a.equals(b));          // true  → equals "yazılar aynı mı?" sorar. Aynı
System.out.println(a.equalsIgnoreCase(d)); // true → büyük/küçük harfe bakmadan aynı mı?
```

Benzetme: İki ayrı **"Java" yazılı kâğıt**. == "bu **aynı kâğıt** mı?" diye sorar → hayır. equals "üstünde **aynı şey** mi yazıyor?" diye sorar → evet. Yazıları her zaman **equals** ile karşılaştır.

PDF 2 · if ve switch

if = "**eğer** şu doğruysa, şunu yap". Program yol ayrımına gelir ve koşula göre bir yola sapar.

20. Basit if

```java
int age = 20;
if (age > 18) {                               // yaş 18'den büyük mü? (20 > 18 → evet)
    System.out.println("Yaş 18'den büyük");   // evet olduğu için yazılır
}
int temperature = 35;
if (temperature > 30) {                       // 35 > 30 → evet
    System.out.println("Hava sıcak, su için!"); // yazılır
}
if (temperature < 0) {                        // 35 < 0 → hayır
    System.out.println("Don uyarısı!");       // ATLANIR, yazılmaz
}
System.out.println("Program devam ediyor");   // if'ten bağımsız, her zaman yazılır
```

Benzetme: "Yağmur yağıyorsa şemsiye al." Yağmur yoksa hiçbir şey yapma, yoluna devam et.

21. if-else

```java
int number = 13;
if (number % 2 == 0) {                        // 13'ü 2'ye bölünce kalan 0 mı? (kalan 1 → hayır)
    System.out.println(number + " çift sayıdır");
} else {                                      // değilse buraya gel
    System.out.println(number + " tek sayıdır"); // 13 tek sayıdır
}
```

Benzetme: "Açsan ye, değilsen (else) oyun oyna." İki yoldan biri mutlaka seçilir.

22. Artık yıl kontrolü

```java
int[] years = {2020, 2023, 1900, 2000};          // kontrol edilecek yıllar
for (int year : years) {                         // her yılı tek tek al
    boolean leap = (year % 4 == 0 && year % 100 != 0)   // 4'e bölünür VE 100'e bölünmez
                 || (year % 400 == 0);                  // VEYA 400'e bölünür
    if (leap) {
        System.out.println(year + " -> LEAP YEAR");     // artık yıl
    } else {
        System.out.println(year + " -> COMMON YEAR");   // normal yıl
    }
}
// 2020 LEAP · 2023 COMMON · 1900 COMMON (400'e bölünmüyor) · 2000 LEAP
```

Benzetme: Uzun koşulu önce **leap** adlı bir kutuya koyduk; if sadece kutuya bakıyor. Okunması kolay.

23. if-else-if merdiveni (not sistemi)

```java
int marks = 65;
if (marks < 0 || marks > 100) {          // önce saçma notları ele
    System.out.println("Geçersiz not!");
} else if (marks >= 90) {                // 65 ≥ 90? hayır → aşağı in
    System.out.println("A+ grade");
} else if (marks >= 80) {                // hayır → in
    System.out.println("A grade");
} else if (marks >= 70) {                // hayır → in
    System.out.println("B grade");
} else if (marks >= 60) {                // 65 ≥ 60 → EVET → bunu yaz ve merdivenden çık
    System.out.println("C grade");
} else if (marks >= 50) {                // artık bakılmaz
    System.out.println("D grade");
} else {
    System.out.println("fail");
}
// Çıktı: C grade
```

Benzetme: Merdivenden inmek: her basamakta "bu mu?" diye sorarsın, **ilk evet** dediğin basamakta durursun. Alttakilere bakmazsın.

24. Sıra yanlışsa ne olur?

```java
int marks = 95;
if (marks >= 50) {                       // 95 ≥ 50 → EVET → hemen burada durur
    System.out.println("D grade");       // YANLIŞ: D grade yazar
} else if (marks >= 90) {                // buraya hiç gelinmez
    System.out.println("A+ grade");
}
// Doğrusu: önce >= 90, sonra >= 50 kontrol et → A+ grade
```

Benzetme: Kapıda "18 yaş üstü mü?" diye sorup herkesi içeri alırsan, **VIP**'leri ayırt edemezsin. Önce en dar koşulu sor.

25. Pozitif / negatif / sıfır

```java
int[] numbers = {-13, 0, 42};
for (int number : numbers) {
    if (number > 0) {                                   // sıfırdan büyükse
        System.out.println(number + ": POSITIVE");
    } else if (number < 0) {                            // sıfırdan küçükse
        System.out.println(number + ": NEGATIVE");
    } else {                                            // ikisi de değilse → sıfırdır
        System.out.println(number + ": ZERO");
    }
}
// -13: NEGATIVE · 0: ZERO · 42: POSITIVE
```

26. İç içe if (nested if)

```java
int age = 25;
int weight = 48;
if (age >= 18) {                                        // 1. kapı: yaş 18+ mı? evet
    if (weight > 50) {                                  // 2. kapı: kilo 50 üstü mü? hayır
        System.out.println("Kan bağışı yapabilirsiniz");
    } else {
        System.out.println("Kilo yetersiz (50 kg üstü)"); // bu yazılır
    }
} else {
    System.out.println("Yaş en az 18 olmalı");
}
```

Benzetme: Havalimanı: önce pasaport kontrolü, geçersen içeride bagaj kontrolü. İlk kapıdan geçemezsen ikinciye hiç gelmezsin.

Kısa yolu (&&):

```java
int age = 20, weight = 80;
if (age >= 18 && weight > 50) {      // ikisi de doğruysa
    System.out.println("Uygun");      // Uygun
}
```

Her koşula ayrı mesaj gerekiyorsa iç içe if, tek sonuç varsa && kullan.

27. if-else yerine ternary

```java
int number = 13;
String output = (number % 2 == 0) ? "çift" : "tek";   // tek satırda if-else
System.out.println(output);                            // tek
```

28. if tuzakları

```java
int x = 3;
if (x > 5);                                    // DİKKAT: buradaki ; if'i BİTİRDİ
{
    System.out.println("A: Her zaman çalışır");   // if'e bağlı değil → yazılır
}
if (x > 5)                                     // süslü parantez YOK
    System.out.println("B: koşula bağlı");     // sadece BU satır if'e bağlı → yazılmaz
    System.out.println("C: if'e bağlı DEĞİL"); // içeride görünse de değil → yazılır
// if (x = 5) { }  → HATA: = atamadır, karşılaştırma == ile
// if (x) { }      → HATA: koşul true/false olmalı, sayı olmaz
```

Benzetme: if (...); = **kapıyı açıp hemen kapatmak**. Alttaki oda artık kapıya bağlı değil, herkes girer. Kural: **her zaman { } kullan.**

29. switch (ay numarasından ay adı)

```java
int month = 7;
String monthName;
switch (month) {                               // month'un değerine bak
    case 1: monthName = "Ocak"; break;         // 1 ise Ocak, break = switch'ten çık
    case 2: monthName = "Şubat"; break;
    // ... 3-6 aynı şekilde
    case 7: monthName = "Temmuz"; break;       // 7 → Temmuz → çık
    // ... 8-12 aynı şekilde
    default: monthName = "Geçersiz ay!";       // hiçbiri tutmazsa
}
System.out.println(month + " - " + monthName);   // 7 - Temmuz
```

Benzetme: switch = **asansör düğmeleri**. Hangi numaraya basarsan o kata gidersin. default = "böyle kat yok" uyarısı. break = **asansörden in**.

30. break unutulursa (fall-through)

```java
int number = 20;
switch (number) {
    case 10: System.out.println("10");     // 20 ≠ 10, atla
    case 20: System.out.println("20");     // EŞLEŞTİ → yaz
    case 30: System.out.println("30");     // break yok → bunu da yaz!
    default: System.out.println("Diğer");  // break yok → bunu da yaz!
}
// Çıktı: 20 · 30 · Diğer
```

Benzetme: Domino taşı: eşleşen yerden itibaren break görene kadar hepsi devrilir.

31. Bilerek fall-through (gruplama)

```
char ch = 'O';
switch (Character.toLowerCase(ch)) {          // harfi küçült: 'O' → 'o'
    case 'a': case 'e': case 'ı': case 'i':   // bu harflerden biriyse...
    case 'o': case 'ö': case 'u': case 'ü':   // ...hepsi aynı satıra düşer
        System.out.println("Vowel (ünlü)");   // Vowel (ünlü)
        break;
    default:
        System.out.println("Consonant");      // diğer harfler ünsüz
}
```

32. switch ile yazı (String)

```
String levelString = "Expert";
int level;
switch (levelString) {
    case "Beginner": level = 1; break;
    case "Intermediate": level = 2; break;
    case "Expert": level = 3; break;          // eşleşti → level = 3
    default: level = 0;
}
System.out.println("Your Level is: " + level);   // Your Level is: 3
String cmd = "EXIT";
switch (cmd.toLowerCase()) {                      // "EXIT" → "exit" (harf farkını sil)
    case "exit": System.out.println("Kapanıyor"); break;   // Kapanıyor
    default: System.out.println("Bilinmeyen komut");
}
```

Dikkat: "EXIT" ile "exit" switch için **farklı**; o yüzden toLowerCase() ile küçülttük.

33. switch ile enum

```
enum Day { SUN, MON, TUE, WED, THU, FRI, SAT }   // haftanın günleri listesi
for (Day d : Day.values()) {                      // values() = listedeki tüm günler sırayla
    switch (d) {
        case SAT:                                 // Cumartesi...
        case SUN:                                 // ...veya Pazar ise
            System.out.println(d + ": Hafta sonu");
            break;
        default:                                  // diğer günler
            System.out.println(d + ": Hafta içi");
    }
}
```

Dikkat: case'te **Day.SAT değil, sadece SAT** yazılır.

34. Modern switch (ok ->)

```
int day = 6;
String type = switch (day) {                // switch bir DEĞER üretiyor, type'a koyuluyor
    case 1, 2, 3, 4, 5 -> "Hafta içi";      // virgülle birden fazla değer
    case 6, 7 -> "Hafta sonu";              // 6 → "Hafta sonu"
    default -> "Geçersiz";
};
System.out.println(type);                   // Hafta sonu
char grade = 'B';
switch (grade) {
    case 'A' -> System.out.println("Mükemmel");
    case 'B', 'C' -> System.out.println("İyi");   // İyi
    default -> System.out.println("Gelişmeli");
}
```

Benzetme: Ok (->) = **otomatik break**. Domino devrilmez, sadece eşleşen satır çalışır.

PDF 2 · for ve for-each

for = "bu işi **şu kadar kez** tekrarla". Kaç kez döneceğini baştan biliyorsan for kullan.

```
for (int i = 1; i <= 5; i++) { ... }
//   ①başla    ②devam mı?  ④her tur sonunda artır     ③yapılacak iş
```

Parça	Basit anlamı
int i = 1	Sayacı 1'den başlat (sadece bir kez)
i <= 5	Sayaç 5'i geçmedikçe devam et (her turdan önce sorulur)
i++	Her tur bitince sayacı 1 artır
{ ... }	Her turda yapılacak iş

Benzetme: Spor salonunda **"5 tur koş"**: 1. turdan başla, 5'e kadar devam, her tur bitince sayacı bir artır.

35. Temel for örnekleri

```
for (int i = 1; i <= 10; i++) {          // i: 1, 2, 3 ... 10
    System.out.print(i + " ");           // 1 2 3 4 5 6 7 8 9 10 (aynı satıra)
}
System.out.println();                    // alt satıra geç

int number = 5, factorial = 1;           // factorial'ı 1'den başlat (çarpımda 0 olmaz!)
for (int i = 1; i <= number; i++) {
    factorial *= i;                      // 1x1=1, 1x2=2, 2x3=6, 6x4=24, 24x5=120
}
System.out.println(number + "! = " + factorial);   // 5! = 120

int sum = 0;                             // toplamı 0'dan başlat
for (int i = 2; i <= 100; i += 2) {      // i: 2, 4, 6 ... 100 (2'şer atla)
    sum += i;                            // her çift sayıyı ekle
}
System.out.println("Çift sayılar toplamı: " + sum);   // 2550
```

Benzetme: sum ve factorial = **kumbara**. Döngüden önce boş kumbara al (toplamda 0, çarpımda 1), her turda içine at.

36. for çeşitleri

```
for (int i = 10; i >= 1; i -= 3) {       // 10'dan geriye 3'er say
    System.out.print(i + " ");           // 10 7 4 1
}
for (int i = 0, j = 10; i < j; i++, j--) {   // iki sayaç: i artıyor, j azalıyor
    System.out.print(i + "-" + j + " ");     // 0-10 1-9 2-8 3-7 4-6 (ortada buluşunca dur)
}
int k = 1;
for (; k < 100; ) {                      // başlangıç ve artırma kısmı boş bırakılabilir
    k *= 3;                              // 1 → 3 → 9 → 27 → 81 → 243
}
System.out.println("k = " + k);          // k = 243
for (char c = 'a'; c <= 'e'; c++) {      // harflerle de sayılır
    System.out.print(c);                 // abcde
}
```

37. İç içe for: çarpım tablosu

```
for (int i = 1; i <= 5; i++) {           // DIŞ döngü: satırlar (1'den 5'e)
    for (int j = 1; j <= 5; j++) {       // İÇ döngü: her satırdaki sütunlar
        System.out.printf("%4d", i * j); // satır x sütun, 4 karakterlik yere yaz
    }
    System.out.println();                // satır bitti, alta geç
}
```

Benzetme: Saat: yelkovan (iç döngü) bir tam tur atınca akrep (dış döngü) bir ilerler. İç döngü her dış turda baştan sona çalışır: 5 x 5 = 25 kez.

38. Yıldız desenleri

```java
// Üçgen: 1. satırda 1 yıldız, 2. satırda 2 yıldız...
for (int i = 1; i <= 5; i++) {
    for (int j = 1; j <= i; j++) {       // satır numarası kadar yıldız
        System.out.print("* ");
    }
    System.out.println();
}
// Ters üçgen: her satırda bir eksik
int term = 5;
for (int i = 1; i <= term; i++) {
    for (int j = term; j >= i; j--) {    // 5'ten i'ye kadar geri say → 5, 4, 3, 2, 1 yıldız
        System.out.print("* ");
    }
    System.out.println();
}
```

39. Asal sayı ve Fibonacci

```java
int n = 29;
boolean isPrime = n > 1;                 // 1 ve altı asal değil; 29 > 1 → şimdilik "asal" say
for (int i = 2; i * i <= n; i++) {       // 2'den √29'a (yaklaşık 5) kadar dene
    if (n % i == 0) {                    // tam bölünüyor mu?
        isPrime = false;                 // bölünüyorsa asal DEĞİL
        break;                           // daha fazla aramaya gerek yok, çık
    }
}
System.out.println(n + " asal mı? " + isPrime);   // 29 asal mı? true

int a = 0, b = 1;                        // ilk iki terim
System.out.print("Fibonacci: ");
for (int i = 0; i < 10; i++) {           // 10 terim yaz
    System.out.print(a + " ");           // şimdiki terimi yaz
    int next = a + b;                    // sonraki = son iki terimin toplamı
    a = b;                               // bir adım kay
    b = next;
}
// Fibonacci: 0 1 1 2 3 5 8 13 21 34
```

Benzetme: Asal kontrolü = **kilide anahtar denemek**: bir anahtar uyarsa (bölünürse) kilit asal değildir, diğerlerini denemeye gerek yok (break).

40. Etiketli break

```java
aa:                                      // dış döngüye "aa" adını verdik
for (int i = 1; i <= 3; i++) {
    bb:                                  // iç döngüye "bb" adını verdik
    for (int j = 1; j <= 3; j++) {
        if (i == 2 && j == 2) {
            break aa;                    // aa'yı (DIŞ döngüyü) tamamen bitir
        }
        System.out.println(i + " " + j);
    }
}
// break aa → 1 1, 1 2, 1 3, 2 1 (sonra her şey biter)
// break bb → 1 1, 1 2, 1 3, 2 1, 3 1, 3 2, 3 3 (sadece o iç tur biter)
```

Benzetme: break aa = **binadan çık**. break bb = **sadece odadan çık**, binada devam et.

41. for-each: diziyi gezmek

```java
int[] arr = {12, 23, 44, 56, 78};
for (int x : arr) {                      // "arr'daki her x için" → x sırayla 12, 23, 44...
    System.out.print(x + " ");           // 12 23 44 56 78
}
int total = 0, max = arr[0];             // max'ı ilk elemanla başlat
for (int x : arr) {
    total += x;                          // hepsini topla
    if (x > max) {                       // daha büyüğünü görürsen
        max = x;                         // max'ı güncelle
    }
}
System.out.println("Toplam: " + total);  // 213
System.out.println("Maksimum: " + max);  // 78
double avg = (double) total / arr.length;  // (double) olmasa 42 çıkardı
System.out.println("Ortalama: " + avg);  // 42.6
```

Benzetme: for-each = **sepetten elmaları tek tek almak**. Kaçıncı elma olduğunu (index) umursamazsın, sadece elmayla ilgilenirsin.

42. for-each: liste ve 2 boyutlu dizi

```
import java.util.ArrayList;                    // ArrayList'i kullanmak için içeri al
ArrayList<String> list = new ArrayList<>();    // büyüyebilen yazı listesi
list.add("Ali");                               // listeye ekle
list.add("Ayşe");
list.add("Mehmet");
for (String name : list) {                     // listedeki her isim için
    System.out.println(name + " (" + name.length() + ")");   // Ali (3) · Ayşe (4) · Mehmet (6)
}
int[][] matrix = {{1, 2, 3}, {4, 5, 6}};       // 2 satır, 3 sütun tablo
for (int[] row : matrix) {                     // her SATIR (kendisi bir dizi)
    for (int v : row) {                        // satırdaki her sayı
        System.out.print(v + " ");
    }
    System.out.println();                      // 1 2 3 / 4 5 6
}
```

43. for-each diziyi değiştiremez

```
int[] arr = {1, 2, 3};
for (int x : arr) {
    x = x * 10;                          // sadece x'in KOPYASI değişir, dizi aynı kalır
}
System.out.println(Arrays.toString(arr));   // [1, 2, 3]

for (int i = 0; i < arr.length; i++) {   // klasik for: index ile gerçek kutuya git
    arr[i] = arr[i] * 10;                // dizinin kendisini değiştir
}
System.out.println(Arrays.toString(arr));   // [10, 20, 30]
```

Benzetme: for-each sana elmanın **fotokopisini** verir; fotokopiyi boyamak gerçek elmayı boyamaz. Gerçeğini değiştirmek için klasik for ile **rafın numarasına (index)** gitmen lazım. Arrays.toString(arr) = diziyi [1, 2, 3] biçiminde yazdırır.

PDF 3 · while ve do-while

while = "koşul doğru **olduğu sürece** tekrarla". Kaç kez döneceğini bilmiyorsan kullanılır. **Önce sorar, sonra yapar**: koşul baştan yanlışsa hiç çalışmaz.

Benzetme: "Bardak dolana kadar su koy." Kaç kere koyacağını bilmezsin, sadece bardağa bakarsın. Bardak zaten doluysa hiç koymazsın.

44. Temel while

```
int i = 1;                                  // sayacı 1'den başlat
while (i <= 5) {                            // i 5'i geçmedikçe dön
    System.out.println("i = " + i);         // i = 1, i = 2 ... i = 5
    i++;                                    // i'yi artır (UNUTURSAN sonsuz döngü!)
}
System.out.println("Loop bitti, i = " + i);    // Loop bitti, i = 6 (döngüyü bozan ilk değer)
int x = 100;
while (x < 10) {                            // 100 < 10? HAYIR → içeri hiç girmez
    System.out.println("Bu satır hiç yazılmaz");
}
```

45. Sayının basamaklarıyla oynamak

```java
int number = 7351;
int sum = 0, reversed = 0, digits = 0;
int n = number;                          // asıl sayı bozulmasın diye kopyasıyla çalış
while (n > 0) {                          // sayı bitene kadar
    int digit = n % 10;                  // SON basamağı al (7351 % 10 = 1)
    sum += digit;                        // basamakları topla
    reversed = reversed * 10 + digit;    // ters sayıyı kur: 0→1→15→153→1537
    digits++;                            // basamak say
    n /= 10;                             // son basamağı SİL (7351 → 735)
}
System.out.println("Basamak sayısı: " + digits);      // 4
System.out.println("Basamaklar toplamı: " + sum);     // 16
System.out.println("Ters çevrilmiş: " + reversed);    // 1537
```

Tur	n	digit (n % 10)	sum	reversed	n / 10
1	7351	1	1	1	735
2	735	5	6	15	73
3	73	3	9	153	7
4	7	7	16	1537	0 → dur

Benzetme: n % 10 = sayının **kuyruğunu kopar**. n / 10 = **kuyruğu at**. reversed * 10 + digit = koparılan kuyrukları **yeni bir sıraya dizmek**.

46. EBOB ve Collatz

```java
int a = 48, b = 18;
while (b != 0) {              // b sıfır olana kadar
    int temp = b;             // b'yi yedekle
    b = a % b;                // yeni b = kalan (48 % 18 = 12)
    a = temp;                 // yeni a = eski b (18)
}                             // 48,18 → 18,12 → 12,6 → 6,0 → dur
System.out.println("EBOB(48, 18) = " + a);   // 6

int n = 6, steps = 0;
System.out.print(n);
while (n != 1) {                                  // 1'e ulaşana kadar
    n = (n % 2 == 0) ? n / 2 : 3 * n + 1;         // çiftse yarıya böl, tekse 3 katı + 1
    System.out.print(" " + n);
    steps++;                                      // adım say
}
System.out.println("\nAdım sayısı: " + steps);  // 6 3 10 5 16 8 4 2 1 · Adım sayısı: 8
```

Benzetme: Kaç adımda biteceğini **bilemezsin**; o yüzden for değil while kullanılır. \n = alt satıra geç.

47. İç içe while

```java
int i = 1;
while (i <= 5) {                      // dış: satırlar
    int j = 1;                        // HER SATIRDA j'yi sıfırla! (dışarı alırsan sadece 1. satır çalışır)
    while (j <= 5) {                  // iç: sütunlar
        System.out.printf("%4d", i * j);
        j++;
    }
    System.out.println();
    i++;
}
```

48. Sonsuz döngü

```java
int count = 0;
while (count < 5) {
    System.out.println("Count: " + count);   // Count: 0, Count: 0, Count: 0... SONSUZA KADAR
    // count++ unutuldu! count hep 0, koşul hep doğru
}
// Düzeltme: içine count++; ekle → Count: 0 ... Count: 4
```

Benzetme: Arabanın gazına basıp freni hiç kullanmamak. Durdurmak için konsolda **Ctrl+C**.

49. Kullanıcı -1 yazana kadar sayı al (sentinel)

```java
Scanner sc = new Scanner(System.in);          // klavyeden okuyucu (import java.util.Scanner gerekir)
int sum = 0, count = 0;
System.out.print("Sayı girin (-1 = bitir): ");
int n = sc.nextInt();                         // kullanıcıdan bir sayı oku
while (n != -1) {                             // -1 değilse devam
    sum += n;                                 // topla
    count++;                                  // kaç sayı girildi
    System.out.print("Sayı girin (-1 = bitir): ");
    n = sc.nextInt();                         // yeni sayı oku
}
if (count > 0) {                              // hiç sayı girildiyse
    System.out.println("Ortalama: " + (double) sum / count);   // 10, 20, 45 → 25.0
} else {
    System.out.println("Hiç sayı girilmedi");  // 0'a bölmeyi engeller
}
sc.close();                                   // okuyucuyu kapat
```

Benzetme: Sentinel = "Bitti" bayrağı. Garson siparişleri yazar, sen "tamam" (-1) deyince durur.

50. do-while: önce yap, sonra sor

```java
do {
    // iş
} while (koşul);     // sonundaki ; ŞART
```

do-while en az **bir kez** çalışır, çünkü koşula **sonradan** bakar.

```java
int i = 10;
while (i < 5) {                               // önce sor: 10 < 5? hayır → hiç girme
    System.out.println("while: " + i);
    i++;
}
System.out.println("while bitti");             // sadece: while bitti

int k = 10;
do {
    System.out.println("do-while: " + k);      // önce yap: do-while: 10
    k++;
} while (k < 5);                              // sonra sor: 11 < 5? hayır → çık
System.out.println("do-while bitti");
```

Benzetme: while = **"Önce menüye bak, beğenirsen ye."** do-while = **"Önce bir lokma tat, sonra devam edip etmeyeceğine karar ver."**

51. do-while temel örnekler

```java
int i = 1;
do {
    System.out.print(i + " ");            // 1 2 3 4 5 6 7 8 9 10
    i++;
} while (i <= 10);
int number = 5, factorial = 1, k = 1;
do {
    factorial *= k;                       // 1, 2, 6, 24, 120
    k++;
} while (k <= number);
System.out.println(number + "! = " + factorial);   // 5! = 120
```

52. Doğru not girilene kadar sor

```java
Scanner sc = new Scanner(System.in);
int grade;
do {
    System.out.print("Not girin (0-100): ");
    grade = sc.nextInt();                             // notu oku
    if (grade < 0 || grade > 100) {                   // aralık dışıysa
        System.out.println("Geçersiz! Tekrar deneyin.");
    }
} while (grade < 0 || grade > 100);                   // geçersizse başa dön
System.out.println("Kaydedilen not: " + grade);       // 150 ✗, -5 ✗, 85 ✓ → 85
```

Benzetme: Şifre ekranı: doğru şifreyi girene kadar tekrar sorar. En az bir kez sorması gerektiği için do-while.

53. Menü

```java
Scanner sc = new Scanner(System.in);
int choice;
do {
    System.out.println("1) Selamla 2) Yardım 0) Çıkış");   // menüyü göster
    System.out.print("Seçiminiz: ");
    choice = sc.nextInt();                                  // seçimi oku
    switch (choice) {
        case 1 -> System.out.println("Merhaba!");
        case 2 -> System.out.println("Bir numara girin.");
        case 0 -> System.out.println("Güle güle");
        default -> System.out.println("Geçersiz seçim");
    }
} while (choice != 0);                                      // 0 seçilmedikçe menü tekrar gelir
```

Benzetme: ATM menüsü: "Çıkış"a basana kadar menü tekrar tekrar ekrana gelir.

	for	while	do-while
Ne zaman sorar?	Önce	Önce	**Sonra**
En az kaç kez çalışır?	0	0	**1**
Ne zaman kullan?	Kaç tur olacağı belli	Belli değil	Belli değil + en az 1 kez
Örnek	1'den 100'e topla	Dosya bitene kadar oku	Menü, şifre sorma

PDF 3 · break, continue, comments

Komut	En basit anlamı	Benzetme
break	Döngüyü **tamamen** bitir, dışarı çık	Koşudan **tamamen çekilmek**
continue	**Bu turu** atla, sonraki tura geç	Bir turu **pas geçip** koşmaya devam etmek
return	Method'u bitir (gerekirse değer döndür)	İşi bırakıp **eve dönmek**

54. break ile arama

```java
int[] array = {7, 2, 9, 4, 5};
int target = 9;                             // aradığımız sayı
int foundIndex = -1;                        // -1 = "henüz bulunamadı" işareti
for (int i = 0; i < array.length; i++) {
    System.out.println("Kontrol: index " + i);   // index 0, 1, 2
    if (array[i] == target) {               // aradığımızı bulduk mu?
        foundIndex = i;                     // yerini kaydet
        break;                              // BULDUK, aramayı bırak
    }
}
System.out.println("Bulunan index: " + foundIndex);   // 2 (index 3 ve 4'e hiç bakılmadı)
```

Benzetme: Çantada **anahtarı arıyorsun**; bulunca aramayı bırakırsın, çantanın geri kalanını boşaltmazsın.

55. while(true) + break ve do-while + break

```java
int i = 1;
while (true) {                          // sonsuz döngü (bilerek)
    if (i * i > 50) {                   // karesi 50'yi geçtiyse
        break;                          // çık
    }
    System.out.print(i * i + " ");      // 1 4 9 16 25 36 49
    i++;
}
System.out.println("\nSonuç: " + i * i);   // Sonuç: 64 (8x8, döngüyü bitiren)

int k = 1;
do {
    if (k == 5) {
        break;                          // 5'e gelince dur (koşul hâlâ doğru olsa bile)
    }
    System.out.print(k + " ");          // 1 2 3 4
    k++;
} while (k <= 10);
```

56. Etiketli break: iç içe döngüden tamamen çık

```java
int[][] matrix = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
int target = 5;
boolean found = false;
search:                                          // dış döngünün adı "search"
for (int r = 0; r < matrix.length; r++) {        // satırları gez
    for (int c = 0; c < matrix[r].length; c++) { // sütunları gez
        if (matrix[r][c] == target) {
            System.out.println("Bulundu: [" + r + "][" + c + "]");   // Bulundu: [1][1]
            found = true;
            break search;                        // İKİ döngüden birden çık
        }
    }
}
System.out.println("found = " + found);          // found = true
```

Benzetme: Etiketsiz break sadece **içteki odadan** çıkar. break search = **binanın dışına** çıkar.

57. switch içindeki break tuzağı

```java
for (int i = 1; i <= 3; i++) {
    switch (i) {
        case 2:
            System.out.println("2 bulundu");
            break;                   // SADECE switch'ten çıkar, for devam eder
        default:
            System.out.println(i);
    }
}
// 1 · 2 bulundu · 3
// for'dan da çıkmak için: for'a "loop:" adı ver, "break loop;" yaz → 1 · 2 bulundu
```

58. continue: bu turu atla

```java
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        continue;                    // 5'te alttaki satırı ATLA, i++ ile 6'ya geç
    }
    System.out.print(i + " ");       // 1 2 3 4 6 7 8 9 10
}
for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) continue;        // çiftse atla
    System.out.print(i + " ");       // 1 3 5 7 9
}
```

Benzetme: Sınavda bilmediğin soruyu atlamak: sınavdan çıkmıyorsun (break değil), sadece o soruyu geçiyorsun.

59. İç içe continue ve etiketli continue

```java
aa:
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        if (i == 2 && j == 2) {
            continue;        // sadece "2 2"yi atla → 2 1, 2 3 yazılır
            // continue aa;  // dış döngünün sonraki turuna geç → "2 2" ve "2 3" atlanır
        }
        System.out.println(i + " " + j);
    }
}
```

60. while içinde continue tuzağı

```java
int i = 1;
while (i <= 10) {
    if (i == 5) {
        continue;        // ✗ i++'a hiç gelmez → i hep 5 → SONSUZ döngü (çıktı: 1 2 3 4)
    }
    System.out.println(i);
    i++;
}
// ✓ Düzeltme: if (i == 5) { i++; continue; } → 1 2 3 4 6 7 8 9 10
```

Benzetme: for'da i++ otomatik çalışır; while'da i++ **senin elinde**. continue onu atlarsa sayaç hiç ilerlemez.

61. continue ile veri filtreleme (guard clause)

```java
String[] names = {"Ali", null, "Ayşe", "", "Mehmet", null};
int printed = 0;
for (String name : names) {
    if (name == null) continue;      // boş kutuysa (null) atla
    if (name.isEmpty()) continue;    // boş yazıysa ("") atla
    System.out.println(name);        // Ali · Ayşe · Mehmet
    printed++;
}
System.out.println(printed + " isim yazdırıldı");   // 3 isim yazdırıldı
```

Benzetme: Kapıdaki güvenlik görevlisi: uygunsuzları baştan çevirir, içeride sadece doğru kişilerle uğraşılır. Önce null kontrolü: yoksa null.isEmpty() programı çökertir.

62. Yorum (comment) türleri

```java
public class CommentExample {
    public static void main(String[] args) {
        int i = 10;                    // tek satırlık not: bu satırın sonuna kadar
        /* Aşağıdaki kod geçici olarak kapatıldı:
        float j = 5.9f;
        System.out.println(j);
        */                             // /* ile */ arası tamamen kapalı
        System.out.println(i);         // 10
        // System.out.println("test"); → başına // koyunca bu satır çalışmaz
    }
}
```

Benzetme: Yorum = **kodun kenarına kurşun kalemle not almak**. Java bu notları okumaz. Bir kodu silmeden geçici kapatmak için de kullanılır. IDE kısayolu: **Ctrl + /**.

63. Javadoc (belge yorumu)

```java
/**
 * Basit aritmetik işlemler yapan sınıf.    ← sınıfın ne işe yaradığı
 * @author Ad Soyad                         ← yazan kişi
 * @version 1.0                             ← sürüm
 */
public class Calculate {
    /**
     * İki tamsayının toplamını hesaplar.   ← method ne yapıyor
     * @param a birinci sayı                ← 1. parametre ne
     * @param b ikinci sayı                 ← 2. parametre ne
     * @return a ile b'nin toplamı          ← ne geri döndürüyor
     */
    public int sum(int a, int b) {
        return a + b;
    }
    public static void main(String[] args) {
        System.out.println(new Calculate().sum(40, 20));   // 60
    }
}
```

Benzetme: Javadoc = **ürünün kullanım kılavuzu**. javadoc -d docs Calculate.java komutuyla bundan web sayfası şeklinde kılavuz üretilir; IDE'de method'un üzerine gelince bu açıklama çıkar.

64. İyi yorum, kötü yorum

```java
// ✗ KÖTÜ: kodun zaten söylediğini tekrar ediyor
int x = 5;      // x'e 5 ata
x++;            // x'i 1 artır

// ✓ İYİ: NEDEN böyle yaptığını açıklıyor
// Sunucu geçici hata verebilir; 5 denemeden sonra vazgeçiyoruz.
final int MAX_RETRY = 5;
// TODO: deneme aralığını ayarlanabilir yap   ← yapılacak iş notu, IDE listeler
int retryCount = 0;
```

65. Çalışan yorum (merak konusu)

```java
// \u000d System.out.println("Comment çalıştı!");
// Çıktı: Comment çalıştı!
```

Neden? \u000d gizli bir **"Enter" tuşu**. Java önce onu gerçek alt satıra çevirir, println yorumun dışına düşer ve çalışır. Gerçek kodda asla kullanma.

PDF 4 · Class ve object

Kavram	En basit anlamı	Benzetme
class	Kalıp, plan	**Kurabiye kalıbı**
object	Kalıptan üretilen gerçek şey	**Kurabiyenin kendisi**
field	Object'in bilgileri	Kurabiyenin rengi, tadı
method	Object'in yapabildikleri	"Ye", "paketle"
new	Kalıptan yeni bir tane bas	Kalıbı hamura bastırmak

Bir kalıptan istediğin kadar kurabiye çıkar; her birinin kendi rengi olabilir.

66. İlk class ve object

```
class Dog {                                  // Köpek KALIBI
    String name;                             // her köpeğin bir adı olacak (bilgi)
    void bark() {                            // her köpek havlayabilir (davranış)
        System.out.println(name + " says Woof!");
    }
}
public class Main {
    public static void main(String[] args) {
        Dog myDog = new Dog();               // kalıptan 1. köpeği üret
        myDog.name = "Rocky";                // 1. köpeğin adı Rocky
        myDog.bark();                        // Rocky says Woof!
        Dog other = new Dog();               // kalıptan 2. köpeği üret
        other.name = "Karabaş";              // 2. köpeğin adı Karabaş
        other.bark();                        // Karabaş says Woof!
    }
}
```

Benzetme: Dog = **köpek nüfus cüzdanı şablonu**. new Dog() = **yeni bir köpek doğdu**. Nokta (.) = "...'nın": myDog.name = myDog'un adı.

67. Inheritance (miras) ilk bakış

```
class Vehicle {                                    // ANA kalıp: Taşıt
    void start() { System.out.println("Vehicle is starting..."); }
    void stop() { System.out.println("Vehicle is stopping..."); }
}
class Car extends Vehicle {                        // extends = Car, Vehicle'ın her şeyini ALIR
    void honk() { System.out.println("Car is honking!"); }   // + kendi özelliği
}
Car myCar = new Car();
myCar.start();   // Vehicle is starting...  → babadan geldi
myCar.stop();    // Vehicle is stopping...  → babadan geldi
myCar.honk();    // Car is honking!         → kendi yazdığı
```

Benzetme: Çocuk babasının **gözlerini ve boyunu** miras alır, üstüne **kendi yeteneğini** ekler. Aynı kodu tekrar yazmazsın.

68. Polymorphism (çok biçimlilik)

```
class Animal { void sound() { System.out.println("..."); } }
class Dog extends Animal {
    @Override void sound() { System.out.println("Hav hav"); }   // babanınkini EZ, kendi sesini koy
}
class Cat extends Animal {
    @Override void sound() { System.out.println("Miyav"); }
}
Animal[] animals = { new Dog(), new Cat(), new Animal() };   // hepsi "hayvan" listesinde
for (Animal a : animals) {
    a.sound();       // Hav hav · Miyav · ... → aynı komut, herkes kendi sesini çıkarır
}
```

Benzetme: Öğretmen sınıfa **"Ses çıkarın!"** der. Aynı komut, ama köpek havlar, kedi miyavlar.

69. Abstraction (soyutlama)

```java
abstract class Animal {                     // abstract = yarım kalıp, tek başına kullanılamaz
    abstract void makeSound();              // gövdesiz: "ses çıkarılacak" ama NASIL belli değil
    void breathe() {                        // bu kısım hazır
        System.out.println("Animal is breathing...");
    }
}
class Dog extends Animal {
    @Override
    void makeSound() { System.out.println("Dog barking"); }   // NASIL'ı çocuk dolduruyor
}
Animal myDog = new Dog();
myDog.breathe();     // Animal is breathing...
myDog.makeSound();   // Dog barking
// Animal a = new Animal();  → HATA: yarım kalıptan ürün çıkmaz
```

Benzetme: Telefonla arama yapmak: numarayı tuşlarsın, içeride sinyalin nasıl gittiğini bilmezsin. Abstract class = **"bu işi yapacaksın ama nasıl yapacağına sen karar ver"** diyen sözleşme.

70. Encapsulation (kapsülleme)

```java
class BankAccount {
    private double balance;                     // private = dışarıdan KİMSE dokunamaz
    public void deposit(double amount) {        // para yatırma kapısı
        if (amount > 0) {                       // kontrol: eksi para yatırılmaz
            balance += amount;
        } else {
            System.out.println("Geçersiz miktar!");
        }
    }
    public double getBalance() { return balance; }   // bakiyeyi SADECE göstermek için
}
BankAccount acc = new BankAccount();
acc.deposit(500);                   // bakiye 500
acc.deposit(-100);                  // Geçersiz miktar!
// acc.balance = 1000000;           → HATA: private, doğrudan değiştiremezsin
System.out.println("Bakiye: " + acc.getBalance());   // Bakiye: 500.0
```

Benzetme: Banka kasası: paraya doğrudan elini sokamazsın, **gişeden** (method) işlem yaparsın; gişe görevlisi kontrol eder.

71. Aggregation ve composition

```java
// Aggregation: parça DIŞARIDAN verilir, bağımsız yaşar
class Department {
    String name;
    Department(String n) { name = n; }
}
class University {
    Department dept;                        // üniversitenin bir bölümü VAR
    University(Department d) { dept = d; }   // bölüm dışarıda üretilip veriliyor
}
// Composition: parça İÇERİDE üretilir, bütünle birlikte yaşar
class Engine {
    void start() { System.out.println("Motor çalıştı"); }
}
class Car {
    private final Engine engine = new Engine();   // motor arabayla BİRLİKTE doğuyor
    void drive() { engine.start(); }
}
```

Benzetme: Aggregation = **öğretmen ve okul**: okul kapansa öğretmen başka okulda çalışır. Composition = **insan ve kalp**: kalp insandan ayrı yaşayamaz.

72. Field'ların varsayılan değerleri

```java
class Student {
    int id;             // sayı → değer verilmezse 0
    String name;        // yazı (reference) → null
    double gpa;         // ondalık → 0.0
    boolean active;     // boolean → false
}
Student s1 = new Student();
System.out.println(s1.id);      // 0
System.out.println(s1.name);    // null
System.out.println(s1.gpa);     // 0.0
System.out.println(s1.active);  // false
```

Benzetme: Yeni doğan bebeğin formu: doldurulmamış kutulara Java kendisi **"boş" değerini** yazar.

73. Object'e değer vermenin yolları

```
class Student {
    int rollno;
    String name;
    void insertRecord(int r, String n) {      // değerleri method ile koyan kapı
        rollno = r;
        name = n;
    }
    void displayInformation() {
        System.out.println(rollno + " " + name);
    }
}
Student s1 = new Student();
s1.rollno = 101;                 // 1. yol: doğrudan kutuya koy
s1.name = "Ali";
Student s2 = new Student();
s2.insertRecord(222, "Ayşe");    // 2. yol: method ile koy
s1.displayInformation();         // 101 Ali
s2.displayInformation();         // 222 Ayşe
// 3. yol: constructor (aşağıda)
```

74. Reference: aynı object'e iki etiket

```
class Point { int x, y; }
Point p1 = new Point();
p1.x = 5;
Point p2 = p1;                    // YENİ nokta YOK, p2 de aynı noktayı gösteriyor
p2.x = 99;                        // p2 üzerinden değiştir
System.out.println(p1.x);         // 99 → p1 de değişti, çünkü AYNI nokta
Point p3 = new Point();           // bu gerçekten yeni nokta
p3.x = 99;
System.out.println(p1 == p2);     // true  → aynı nokta mı? evet
System.out.println(p1 == p3);     // false → değerler aynı ama farklı noktalar
```

Benzetme: p2 = p1 = **aynı eve ikinci anahtar** vermek. p2 ile içeri girip duvarı boyarsan, p1 de boyalı duvar görür.

75. Banka hesabı

```
class Account {
    int accNo;
    String name;
    double amount;
    void insert(int a, String n, double amt) {     // hesap aç
        accNo = a; name = n; amount = amt;
    }
    void deposit(double amt) {                     // para yatır
        amount += amt;
        System.out.println(amt + " yatırıldı");
    }
    void withdraw(double amt) {                    // para çek
        if (amount < amt) System.out.println("Yetersiz bakiye");   // yetmiyorsa uyar
        else {
            amount -= amt;
            System.out.println(amt + " çekildi");
        }
    }
    void checkBalance() { System.out.println("Bakiye: " + amount); }
}
// a1.insert(832345, "Ankit", 1000) → checkBalance() → Bakiye: 1000.0
// deposit(40000) → 40000.0 yatırıldı → withdraw(15000) → 15000.0 çekildi
// withdraw(50000) → Yetersiz bakiye → checkBalance() → Bakiye: 26000.0
```

76. Anonymous (isimsiz) object

```java
class Calculation {
    void fact(int n) {
        int fact = 1;
        for (int i = 1; i <= n; i++) fact *= i;
        System.out.println("factorial is " + fact);
    }
}
Calculation c = new Calculation();    // isimli object
c.fact(4);                            // factorial is 24
new Calculation().fact(5);            // İSİMSİZ: üret, bir kez kullan, at → factorial is 120
Calculation c1 = new Calculation(), c2 = new Calculation();   // tek satırda iki object
c1.fact(3);                           // factorial is 6
```

Benzetme: İsimsiz object = **tek kullanımlık bardak**. Kullanırsın, çöpe gider (garbage collector toplar).

PDF 4 · Method ve constructor

Method = adı olan, çağrılınca çalışan **iş paketi**. **Constructor** = object **doğarken** otomatik çalışan özel kurulum kodu.

77. Method'un parçaları

```java
public static int add(int a, int b) { return a + b; }
// public   → herkes kullanabilir
// static   → object üretmeden çağrılabilir
// int      → geri ne döndürecek? bir tam sayı (hiçbir şey dönmüyorsa: void)
// add      → method'un adı (fiil olmalı)
// (int a, int b) → içeri alacağı malzemeler (parametreler)
// { return a + b; } → yapacağı iş: a ile b'yi topla ve geri ver
```

Benzetme: Method = **meyve sıkacağı**. İçine malzeme atarsın (parametre: portakal), çalışır, sana bir şey verir (return: meyve suyu). void ise sana bir şey vermez, sadece işini yapar (ör. ekrana yazar).

78. Erişim belirleyiciler (kim kullanabilir?)

Kelime	Kim kullanabilir?	Benzetme
public	Herkes	Parktaki bank
protected	Aynı paket + miras alan çocuklar	Aile evi: aile ve çocuklar
(hiçbiri yazılmazsa) default	Sadece aynı paket	Apartman ortak alanı
private	Sadece kendi class'ı	Yatak odan, kilitli günlük

79. Hazır (built-in) method'lar

```java
System.out.println(Math.pow(2, 5));    // 32.0 → 2 üzeri 5
System.out.println(Math.sqrt(144));    // 12.0 → karekök
System.out.println(Math.max(17, 42));  // 42   → büyük olanı
System.out.println(Math.abs(-7.5));    // 7.5  → eksiyi at (mutlak değer)
System.out.println(Math.round(4.6));   // 5    → yuvarla
String s = "Java";
System.out.println(s.length());        // 4    → harf sayısı
System.out.println(s.charAt(0));       // J    → 0. sıradaki harf (sayma 0'dan başlar)
System.out.println(s.toUpperCase());   // JAVA → büyük harf
System.out.println(s.equals("java"));  // false → harf büyüklüğü farklı
System.out.println(Integer.parseInt("25") + 5);   // 30 → "25" yazısını sayıya çevir, 5 ekle
```

Benzetme: Hazır method = **mutfaktaki hazır aletler** (blender, tost makinesi). Nasıl yapıldığını bilmeden kullanırsın.

80. static method vs normal method

```java
public class Main {
    static int square(int x) {                    // static: object gerekmez
        return x * x;
    }
    void greet(String name) {                     // normal: object gerekir
        System.out.println("Merhaba " + name);
    }
    public static void main(String[] args) {
        System.out.println(square(6));            // 36 → doğrudan çağır
        System.out.println(Main.square(3));       // 9  → class adıyla da olur
        Main obj = new Main();                    // önce object üret
        obj.greet("Ayşe");                        // Merhaba Ayşe → object üzerinden çağır
        // greet("Ali");                          → HATA: static main, object olmadan normal method'u çağıramaz
    }
}
```

Benzetme: static method = **sokaktaki ortak çeşme**: herkes doğrudan su alır. Normal method = **evdeki musluk**: önce ev (object) lazım.

81. Parametre ve return

```java
static boolean isEven(int n) {             // sayı al, true/false döndür
    return n % 2 == 0;                     // çiftse true
}
static double average(int[] nums) {        // dizi al, ortalama döndür
    int sum = 0;
    for (int v : nums) sum += v;           // hepsini topla
    return (double) sum / nums.length;     // böl ve geri ver
}
static void printLine(int len) {           // void: geri bir şey VERMEZ
    for (int i = 0; i < len; i++) System.out.print("-");
    System.out.println();
}
System.out.println(isEven(10));                      // true
System.out.println(average(new int[]{3, 4, 8}));     // 5.0
printLine(12);                                       // ------------
```

Kural: return = **"al bu sonuç, ben bittim"**: değeri verir ve method'u **hemen** bitirir.

82. Java her zaman kopya gönderir

```java
static void change(int x) {
    x = 100;                        // sadece KOPYAYI değiştirir
}
static void changeArray(int[] a) {
    a[0] = 100;                     // kopya ADRES, aynı diziye gider → gerçek dizi değişir
}
int n = 5;
change(n);
System.out.println(n);              // 5 → orijinal değişmedi
int[] arr = {5, 6};
changeArray(arr);
System.out.println(arr[0]);         // 100 → dizi değişti
```

Benzetme: Sayı göndermek = **fotokopi vermek**: karalasalar senin aslın temiz kalır. Dizi göndermek = **ev adresinin fotokopisini vermek**: kâğıt kopya ama adres aynı ev; gidip evi boyarlarsa senin evin boyanır.

83. Constructor kuralları

Kural	Açıklama
Adı	Class adının **aynısı**
Return type	**Yok**, void bile yazılmaz
Ne zaman çalışır?	new ile object doğarken **otomatik**
Hiç yazmazsan?	Java boş bir tane ekler (default constructor)

Benzetme: Constructor = **araba fabrikadan çıkarken** yapılan ilk ayarlar (renk, motor). Sen çağırmazsın, üretimde kendiliğinden olur.

84. void yazarsan constructor olmaz

```java
class Test {
    void Test() {                                   // void yazıldı → bu artık sıradan METHOD
        System.out.println("Ben bir method'um");
    }
}
Test t = new Test();      // hiçbir şey yazmaz (constructor yok gibi)
t.Test();                 // Ben bir method'um → elle çağırınca çalışır

class Test2 {
    Test2() {                                       // return type YOK → gerçek constructor
        System.out.println("Constructor çalıştı");
    }
}
Test2 t2 = new Test2();   // Constructor çalıştı → kendiliğinden
```

85. Default ve no-arg constructor

```java
class Bike {
    Bike() {                                        // parametresiz constructor
        System.out.println("Bike is created");
    }
}
class Student {                                     // constructor yazılmadı → Java boş bir tane ekler
    int id;
    String name;
}
Bike b = new Bike();                                // Bike is created
Student s = new Student();
System.out.println(s.id + " " + s.name);            // 0 null (varsayılan değerler)
```

86. Parametreli constructor ve overloading

```java
class Student {
    int id, age;
    String name;
    Student(int id, String name) {                  // 2 malzemeli kurulum
        this.id = id;                               // this.id = object'in id'si, id = gelen değer
        this.name = name;
    }
    Student(int id, String name, int age) {         // 3 malzemeli kurulum
        this(id, name);                             // üstteki constructor'ı çağır (tekrar yazma)
        this.age = age;                             // sadece fazlasını ekle
    }
    void display() { System.out.println(id + " " + name + " " + age); }
}
new Student(111, "Karan").display();                // 111 Karan 0 (yaş verilmedi → 0)
new Student(222, "Aryan", 25).display();            // 222 Aryan 25
```

Benzetme: Kafeterya: **"sade kahve"** veya **"sütlü + şekerli kahve"** siparişi. Aynı isim (Student), farklı malzeme sayısı.

87. Parametreli yazınca boş constructor kaybolur

```java
class Point {
    int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }   // sadece 2 parametreli var
}
// Point p = new Point();   → HATA: Java artık boş constructor eklemiyor

class Point2 {
    int x, y;
    Point2() { this(0, 0); }                          // çözüm: boş olanı da yaz
    Point2(int x, int y) { this.x = x; this.y = y; }
}
Point2 p = new Point2();
System.out.println(p.x + "," + p.y);                  // 0,0
```

Benzetme: Java şöyle düşünür: **"Sen kendi kurulumunu yazdın, demek ki ne istediğini biliyorsun; ben karışmam."**

88. Copy constructor (kopyalayıcı)

```
class Student {
    int id;
    String name;
    Student(int id, String name) { this.id = id; this.name = name; }
    Student(Student other) {             // başka bir öğrenciyi al
        this.id = other.id;              // onun id'sini kopyala
        this.name = other.name;          // onun adını kopyala
    }
    void display() { System.out.println(id + " " + name); }
}
Student s1 = new Student(111, "Karan");
Student s2 = new Student(s1);            // s1'in BAĞIMSIZ kopyası
s2.name = "Aryan";                       // sadece kopyayı değiştir
s1.display();                            // 111 Karan → asıl etkilenmedi
s2.display();                            // 111 Aryan
```

Benzetme: s2 = s1 → **aynı eve ikinci anahtar**. new Student(s1) → **aynı planla yeni ev inşa etmek**: birini boyarsan diğeri boyanmaz.

PDF 5 · static ve this

Kelime	En basit anlamı	Benzetme
static	Object'e değil **class'a** ait; herkes için **tek kopya**	Sınıftaki **ortak tahta**
this	**"Ben"**, yani şu anki object'in kendisi	Bir kişinin **"benim adım"** demesi

89. static variable: ortak okul adı

```
class Student {
    int rollno;                          // her öğrencinin kendi numarası
    String name;                         // her öğrencinin kendi adı
    static String college = "ITS";       // OKUL adı herkes için ortak, tek kopya
    Student(int r, String n) { rollno = r; name = n; }
    void display() { System.out.println(rollno + " " + name + " " + college); }
}
Student s1 = new Student(111, "Karan");
Student s2 = new Student(222, "Aryan");
s1.display();                    // 111 Karan ITS
Student.college = "BBDIT";       // okul adını bir kez değiştir...
s1.display();                    // 111 Karan BBDIT
s2.display();                    // 222 Aryan BBDIT → ...herkeste değişti
```

Benzetme: Okulun adı her öğrencinin çantasına ayrı ayrı yazılmaz; **okul kapısında tek bir tabela** var. Tabelayı değiştirirsen herkes yenisini görür.

90. Sayaç: normal mi, static mi?

```
class Counter {
    int count = 0;                // NORMAL: her object kendi sayacını 0'dan başlatır
    Counter() {
        count++;
        System.out.println(count);
    }
}
new Counter(); new Counter(); new Counter();   // 1 · 1 · 1

class Counter2 {
    static int count = 0;         // STATIC: hepsi aynı sayacı paylaşır
    Counter2() {
        count++;
        System.out.println(count);
    }
}
new Counter2(); new Counter2(); new Counter2();   // 1 · 2 · 3
```

Benzetme: Normal = herkesin **kendi tespihi**, herkes 1'den sayar. static = **duvardaki tek sayaç**, herkes aynı sayaca bir ekler.

91. static method

```java
class Student {
    int rollno; String name;
    static String college = "ITS";
    static void change() {           // static method: object gerekmez
        college = "BBDIT";           // ortak tabelayı değiştirir
    }
    Student(int r, String n) { rollno = r; name = n; }
    void display() { System.out.println(rollno + " " + name + " " + college); }
}
class Calculate {
    static int cube(int x) { return x * x * x; }   // yardımcı hesap aracı
}
Student.change();                          // object üretmeden class adıyla çağır
new Student(111, "Karan").display();       // 111 Karan BBDIT
System.out.println(Calculate.cube(5));     // 125
```

Benzetme: static method = **hesap makinesi**: kullanmak için kimsenin evine gitmene gerek yok, doğrudan alıp kullanırsın (Math.max gibi).

92. static method normal değişkene dokunamaz

```java
class A {
    int a = 40;                            // normal (object'e ait) değişken
    public static void main(String[] args) {
        // System.out.println(a);          → HATA: HANGİ object'in a'sı? belli değil
        A obj = new A();                   // önce bir object üret
        System.out.println(obj.a);         // 40 → artık belli: obj'nin a'sı
    }
}
```

Benzetme: Okul müdürü (static) **"öğrencinin notu kaç?"** diye soramaz; önce **hangi öğrenci** olduğunu söylemesi lazım.

93. static block ve çalışma sırası

```java
public class Main {
    static int max;
    static {                                   // static block: program yüklenirken BİR kez
        System.out.println("1) static block");
        max = 100;
    }
    {                                          // instance block: her new'de, constructor'dan ÖNCE
        System.out.println("3) instance block");
    }
    Main() {                                   // constructor
        System.out.println("4) constructor");
    }
    public static void main(String[] args) {
        System.out.println("2) main, max = " + max);
        new Main();
    }
}
// 1) static block · 2) main, max = 100 · 3) instance block · 4) constructor
```

Benzetme: static block = **dükkanı sabah açarken** ışıkları yakmak (bir kez). Instance block + constructor = **her müşteri geldiğinde** yapılan karşılama.

94. static nested class

```java
class Library {
    static int bookCount = 0;              // kütüphanedeki toplam kitap sayısı
    static class Book {                    // kütüphanenin İÇİNDE tanımlanmış kitap kalıbı
        String title;
        Book(String title) {
            this.title = title;
            bookCount++;                   // her yeni kitapta sayacı artır
        }
        void info() { System.out.println("Kitap: " + title); }
    }
}
Library.Book b1 = new Library.Book("Java 101");   // Library object'i üretmeden kitap üret
Library.Book b2 = new Library.Book("OOP");
b1.info();                                 // Kitap: Java 101
System.out.println("Toplam: " + Library.bookCount);   // Toplam: 2
```

95. this ile alan/parametre ayırma

```java
class Student {
    int rollno;
    String name;
    Student(int rollno, String name) {
        rollno = rollno;          // ✗ YANLIŞ: parametreyi kendine atadı, object'in kutusu boş kaldı
        name = name;
    }
}
// Çıktı: 0 null

class Student2 {
    int rollno;
    String name;
    Student2(int rollno, String name) {
        this.rollno = rollno;     // ✓ this.rollno = BENİM kutum, rollno = gelen değer
        this.name = name;
    }
}
// Çıktı: 111 Ankit
```

Benzetme: Odada iki **Ahmet** var: biri misafir (parametre), biri ev sahibi (field). "Ahmet" deyince en yakındaki (misafir) bakar. Ev sahibine seslenmek için **"this.Ahmet"** (bu evin Ahmet'i) demen gerekir.

96. this() ile constructor zinciri

```java
class Student {
    int rollno; String name; float fee;
    Student(int rollno, String name) {
        this.rollno = rollno;
        this.name = name;
    }
    Student(int rollno, String name, float fee) {
        this(rollno, name);          // ÜSTTEKİ constructor'a işin yarısını yaptır
        this.fee = fee;              // sadece ücreti ekle
    }
    void display() { System.out.println(rollno + " " + name + " " + fee); }
}
new Student(111, "ankit").display();          // 111 ankit 0.0
new Student(112, "sumit", 6000f).display();   // 112 sumit 6000.0
```

Kural: this(...) constructor'ın **ilk satırında** olmak zorunda. İkinci satıra yazarsan hata: "call to this must be first statement".

97. this ile kendini göndermek

```java
class Printer {
    void print(Book b) {                              // bir kitap al, yazdır
        System.out.println("Yazdırılıyor: " + b.title);
    }
}
class Book {
    String title;
    Book(String title) { this.title = title; }
    void printWith(Printer p) {
        p.print(this);                                // "BENİ yazdır" → kendini gönder
    }
}
Book book = new Book("Java OOP");
book.printWith(new Printer());                        // Yazdırılıyor: Java OOP
```

Benzetme: Fotokopiciye gidip **"beni fotokopi çek"** demek. this = **kendini teslim etmek**.

98. return this: zincirleme çağrı

```java
class Counter {
    int value;
    Counter add(int n) {
        value += n;
        return this;                // işi yaptım, KENDİMİ geri veriyorum
    }
    Counter reset() {
        value = 0;
        return this;
    }
}
Counter c = new Counter().add(5).add(10).add(2);   // 5 + 10 + 2
System.out.println(c.value);                        // 17
System.out.println(c.reset().add(3).value);         // 3 (sıfırla, 3 ekle)
StringBuilder sb = new StringBuilder();
sb.append("Nesne ").append("Tabanlı ").append("Programlama");   // aynı mantık
System.out.println(sb);                             // Nesne Tabanlı Programlama
```

Benzetme: Tren vagonları: her vagon bir sonrakine bağlanır. add() bitince kendini döndürdüğü için arkasına yeni .add() takılabilir.

99. this gerçekten "ben" mi?

```java
public class Main {
    void check(Main other) {
        System.out.println(this == other);   // ben, gelen kişi miyim?
    }
    void hello() { System.out.println("hello"); }
    void run() {
        hello();                             // aslında this.hello() demek
        this.hello();                        // aynı şey
    }
    public static void main(String[] args) {
        Main obj = new Main();
        obj.check(obj);                      // true → kendisiyle karşılaştırdı
        obj.check(new Main());               // false → başka biriyle
        obj.run();                           // hello · hello
    }
}
```

PDF 5 · Overloading, overriding, super

Kavram	En basit anlamı	Benzetme
Overloading	**Aynı isim**, farklı malzeme (parametre)	"Kahve" sipariş: sade, sütlü, şekerli
Overriding	Çocuk, babasının işini **kendi tarzıyla** yeniden yapar	Babanın tarifini değiştirip kendi yemeğini yapmak
super	**Babaya** ulaşmak	"Baba, sen yap" demek

100. Overloading: sayı, tip, sıra farkı

```java
class Adder {
    static int add(int a, int b) { return a + b; }                // 2 tam sayı
    static int add(int a, int b, int c) { return a + b + c; }     // 3 tam sayı (SAYI farklı)
    static double add(double a, double b) { return a + b; }       // 2 ondalık (TİP farklı)
    static String add(String a, String b) { return a + b; }       // 2 yazı (TİP farklı)
}
class Printer {
    static void show(String s, int n) { System.out.println("S,I"); }   // yazı, sayı
    static void show(int n, String s) { System.out.println("I,S"); }   // sayı, yazı (SIRA farklı)
}
System.out.println(Adder.add(11, 11));          // 22   → 1. add seçildi
System.out.println(Adder.add(11, 11, 11));      // 33   → 2. add
System.out.println(Adder.add(12.5, 12.25));     // 24.75 → 3. add
System.out.println(Adder.add("Java", "!"));     // Java! → 4. add (yazılar yapıştı)
Printer.show("a", 1);                           // S,I
Printer.show(1, "a");                           // I,S
```

Benzetme: Java, **verdiğin malzemeye bakıp** hangi add'in çalışacağına kendisi karar verir. Bu karar program çalışmadan önce (derlerken) verilir.

101. Sadece dönüş tipi farklıysa olmaz

```java
class Adder {
    static int add(int a, int b) { return a + b; }
    static double add(int a, int b) { return a + b; }   // HATA: malzemeler AYNI
}
// add(11, 11) dediğinde Java hangisini seçeceğini bilemez
```

Java'nın kendi println'i overloading'e en güzel örnek: println(5), println('A'), println(2.5), println(true), println("Java"), println() → hepsi aynı isim, farklı malzeme.

102. Tip yükseltme (type promotion)

```java
class Calc {
    void sum(int a, long b) { System.out.println("int,long"); }
    void sum(long a, long b) { System.out.println("long,long"); }
    void show(int a) { System.out.println("int"); }
    void show(double a) { System.out.println("double"); }
}
Calc c = new Calc();
c.sum(20, 20);    // int,long → tam uyan yok, en yakın olan seçildi
c.show(5);        // int      → tam uydu
c.show(5L);       // double   → long'a uyan yok, double'a büyütüldü
c.show('A');      // int      → harf sayıya (65) büyütüldü
c.show(2.5f);     // double   → float double'a büyütüldü
```

Benzetme: Ayakkabı denemek: tam numaran yoksa bir büyüğünü giyersin (byte → short → int → long → float → double). Ama **küçüğünü asla** giymezsin (double → int otomatik olmaz).

103. Kararsızlık (ambiguity)

```java
class Calc {
    void sum(int a, long b) { System.out.println("a"); }
    void sum(long a, int b) { System.out.println("b"); }
}
// new Calc().sum(20, 20);   → HATA: ikisi de eşit derecede uygun, Java seçemez
new Calc().sum(20, 20L);      // a → 2. sayıyı long yaptık, artık tam uyan var
```

Benzetme: İki kapı tam aynı uzaklıktaysa **hangisine gideceğini bilemezsin**. Birini işaretlersen (20L) sorun çözülür.

104. main de overload edilebilir

```java
public class Main {
    public static void main(String[] args) {       // Java SADECE bunu başlatır
        System.out.println("main with String[]");
        main("test");                               // diğerlerini biz çağırırız
        main();
    }
    public static void main(String arg) { System.out.println("main with String"); }
    public static void main() { System.out.println("main without args"); }
}
// main with String[] · main with String · main without args
```

105. Overriding: çocuk kendi versiyonunu yazar

```java
class Vehicle {
    void run() { System.out.println("Vehicle is running"); }
}
class Bike extends Vehicle {
    @Override                                       // "babanınkini eziyorum" etiketi
    void run() { System.out.println("Bike is running"); }   // aynı isim, aynı parametre
}
new Bike().run();     // Bike is running (yazmasaydık: Vehicle is running)
```

Benzetme: Babanın **menemen tarifi** var. Sen aynı adla **kendi menemenini** yaparsın. Senden menemen isteyen seninkini yer.

106. Banka faiz oranları

```java
class Bank {
    int getRateOfInterest() { return 0; }              // genel banka: 0
}
class SBI extends Bank {
    @Override int getRateOfInterest() { return 8; }    // SBI kendi oranını söylüyor
}
class ICICI extends Bank {
    @Override int getRateOfInterest() { return 7; }
}
class AXIS extends Bank {
    @Override int getRateOfInterest() { return 9; }
}
Bank[] banks = { new SBI(), new ICICI(), new AXIS() };     // hepsi "banka" listesinde
for (Bank b : banks) {
    System.out.print(b.getClass().getSimpleName());       // bankanın gerçek adı
    System.out.println(": %" + b.getRateOfInterest());   // her biri KENDİ oranını verir
}
// SBI: %8 · ICICI: %7 · AXIS: %9
```

Benzetme: "Faiz oranınız nedir?" sorusu aynı, ama her banka kendi cevabını verir. Hangi cevabın geleceğine program çalışırken karar verilir.

107. @Override neden önemli?

```java
class Vehicle {
    void run() { System.out.println("Vehicle"); }
}
class Bike extends Vehicle {
    void Run() { System.out.println("Bike"); }   // büyük R → YENİ bir method oldu, ezmedi!
}
Vehicle v = new Bike();
v.run();       // Vehicle → istediğimiz Bike'tı, sessizce yanlış çalıştı
// @Override ekleseydik Java hemen "hiçbir şeyi ezmiyorsun" diye hata verirdi
```

Benzetme: @Override = **imla denetimi**. Yanlış yazdığında anında altını çizer.

108. Erişimi daraltamazsın

```java
class A {
    protected void msg() { System.out.println("Hello java"); }
}
class B extends A {
    // void msg() { }          → HATA: protected'ı daha kapalı (default) yapamazsın
    @Override
    public void msg() { System.out.println("Hello B"); }   // daha açık (public) olur
}
new B().msg();   // Hello B
```

Benzetme: Babanın **herkese açık** dükkânını devralıp **kapısını kilitleyemezsin**; ama daha da açabilirsin.

109. static method ezilmez, gizlenir

```java
class Parent {
    static void hello() { System.out.println("Parent static"); }
    void hi() { System.out.println("Parent instance"); }
}
class Child extends Parent {
    static void hello() { System.out.println("Child static"); }   // sadece GİZLER
    @Override void hi() { System.out.println("Child instance"); } // gerçekten EZER
}
Parent p = new Child();
p.hello();   // Parent static  → static'te etikete (Parent) bakılır
p.hi();      // Child instance → normal method'da gerçek kişiye (Child) bakılır
```

Benzetme: static = **kutunun üzerindeki etiket** neyse o çalışır. Normal method = **kutunun içinde gerçekte ne varsa** o çalışır.

110. super ile babanın alanı ve methodu

```java
class Animal {
    String color = "white";
    void eat() { System.out.println("eating..."); }
}
class Dog extends Animal {
    String color = "black";                          // babanınkiyle aynı isim
    @Override void eat() { System.out.println("eating bread..."); }
    void show() {
        System.out.println(color);           // black   → en yakındaki: benimki
        System.out.println(super.color);     // white   → babanınki
        eat();                               // eating bread... → benimki
        super.eat();                         // eating...       → babanınki
    }
}
new Dog().show();
```

Benzetme: Evde iki **"renk"** var: seninki ve babanınki. Sadece "renk" dersen seninki gelir. **super.renk** dersen babanınkini getirirsin.

111. super(): önce baba kurulur

```java
class Animal {
    Animal() { System.out.println("Animal()"); }
}
class Dog extends Animal {
    Dog() {
        super();                         // önce babanın constructor'ı (yazmasan da Java ekler)
        System.out.println("Dog()");
    }
}
new Dog();   // Animal() · Dog()

class A { A() { System.out.println("A constructor"); } }
class B extends A { B() { System.out.println("B constructor"); } }
class C extends B { C() { System.out.println("C constructor"); } }
new C();     // A constructor · B constructor · C constructor
```

Benzetme: Bina inşaatı: önce temel (dede A), sonra kat (baba B), en son çatı (çocuk C). Çatıyı temelden önce yapamazsın.

112. super(...) ve super.method() birlikte

```java
class Person {
    int id; String name;
    Person(int id, String name) { this.id = id; this.name = name; }
    void display() { System.out.print(id + " " + name); }
}
class Emp extends Person {
    float salary;
    Emp(int id, String name, float salary) {
        super(id, name);                     // id ve adı BABA kursun
        this.salary = salary;                // maaşı ben eklerim
    }
    @Override void display() {
        super.display();                     // önce babanın yazdığını yaz (1 ankit)
        System.out.println(" " + salary);    // üstüne maaşı ekle
    }
}
new Emp(1, "ankit", 45000f).display();       // 1 ankit 45000.0
```

Benzetme: Babanın tarifini çöpe atmıyorsun; **babanın tarifini yapıp üstüne bir sos ekliyorsun**.

113. Babanın boş constructor'ı yoksa

```java
class Person {
    String name;
    Person(String name) { this.name = name; }   // SADECE isimli kurulum var
}
class Emp extends Person {
    // Emp() { }        → HATA: Java gizlice super() ekler ama Person'da boş kurulum yok
    Emp(String name) {
        super(name);    // ✓ babaya istediği ismi ver
    }
}
System.out.println(new Emp("Ali").name);        // Ali
```

PDF 6 · Inheritance türleri ve interface

Inheritance (miras) = bir class'ın başka bir class'ın her şeyini **devralması**. extends ile yapılır. Test cümlesi: **"B bir A'dır"** mantıklıysa miras doğru.

114. Programmer bir Employee'dir

```
class Employee {                                  // BABA: Çalışan
    float salary = 40000;                         // maaş
    void work() { System.out.println("Çalışıyor..."); }
}
class Programmer extends Employee {               // ÇOCUK: Programcı (çalışanın her şeyini alır)
    int bonus = 10000;                            // + kendi bonusu
    void writeCode() { System.out.println("Kod yazıyor..."); }   // + kendi işi
}
Programmer p = new Programmer();
System.out.println("Programmer salary is: " + p.salary);  // 40000.0 → babadan (float olduğu için .0)
System.out.println("Bonus of Programmer is: " + p.bonus); // 10000   → kendisinin
p.work();        // Çalışıyor...   → babadan
p.writeCode();   // Kod yazıyor... → kendisinin
```

Benzetme: Programcı da bir çalışandır; çalışanın **maaşı ve çalışması** ona otomatik geçer, üstüne **bonus ve kod yazma** eklenir.

115. private miras alınır mı?

```
class Account {
    private double balance = 100;      // private: SADECE Account görür
    protected String owner = "Ali";    // protected: çocuklar da görür
    public double getBalance() { return balance; }   // herkese açık kapı
}
class Savings extends Account {
    void show() {
        System.out.println(owner);           // Ali → protected, görebiliyor
        // System.out.println(balance);      → HATA: private, göremez
        System.out.println(getBalance());    // 100.0 → açık kapıdan dolaylı okuyor
    }
}
new Savings().show();
```

Benzetme: Babanın **kasası** (private) evde var ama sen şifresini bilmiyorsun; babana **"ne kadar var?"** diye sorarak (getBalance) öğrenirsin.

116. Her class'ın dedesi: Object

```
class Student {                               // gizlice: extends Object
    String name;
    Student(String name) { this.name = name; }
    @Override
    public String toString() {                // Object'ten gelen method'u değiştir
        return "Student[" + name + "]";
    }
}
Student s = new Student("Ayşe");
System.out.println(s);                        // Student[Ayşe] → println otomatik toString() çağırır
System.out.println(s instanceof Object);      // true → her şey bir Object'tir
System.out.println(s.getClass().getSuperclass().getName());   // java.lang.Object → babası kim?
```

Benzetme: Java'da herkesin **ortak büyük dedesi** var: Object. Bu yüzden her object'te toString(), equals() gibi aile mirası method'lar var. toString yazmasaydık ekranda Student@1b6d3586 gibi anlamsız bir şey çıkardı.

117. Tekli miras (single)

```
class Animal { void eat() { System.out.println("eating..."); } }
class Dog extends Animal { void bark() { System.out.println("barking..."); } }
Dog d = new Dog();
d.bark();   // barking... → kendi
d.eat();    // eating...  → babadan
```

Şekil: Animal → Dog (bir baba, bir çocuk).

118. Çok seviyeli miras (multilevel)

```java
class Animal { void eat() { System.out.println("eating..."); } }
class Dog extends Animal { void bark() { System.out.println("barking..."); } }
class BabyDog extends Dog { void weep() { System.out.println("weeping..."); } }
BabyDog d = new BabyDog();
d.weep();   // weeping... → kendisi
d.bark();   // barking... → babası Dog'dan
d.eat();    // eating...  → dedesi Animal'dan
System.out.println(d instanceof Animal);   // true → yavru köpek de bir hayvandır
```

Şekil: Animal → Dog → BabyDog (**dede → baba → torun**). Torun hepsinin özelliğini taşır.

119. Hiyerarşik miras (hierarchical)

```java
class Animal { void eat() { System.out.println("eating..."); } }
class Dog extends Animal { void bark() { System.out.println("barking..."); } }
class Cat extends Animal { void meow() { System.out.println("meowing..."); } }
Cat c = new Cat();
c.meow();   // meowing...
c.eat();    // eating...
// c.bark();  → HATA: kedi, köpeğin havlamasını bilmez
Animal[] pets = { new Dog(), new Cat() };   // ikisi de "hayvan" listesine girer
for (Animal a : pets) a.eat();              // eating... · eating...
```

Şekil: Animal'dan Dog ve Cat (**iki kardeş**). Kardeşler birbirinin özelliğini almaz, sadece babadan alır.

120. İki babadan miras YOK (diamond problem)

```java
class A { void msg() { System.out.println("Hello"); } }
class B { void msg() { System.out.println("Welcome"); } }
// class C extends A, B { }   → HATA: Java iki class'tan miras vermez
// C'den msg() istesek: Hello mı, Welcome mı? Belli değil → bu yüzden yasak
```

Benzetme: İki baba da sana **"benim soyadımı al"** diyor; hangisini alacağın belli değil. Java bu karmaşayı baştan yasaklıyor.

121. Çözüm: interface

```java
interface Fighter { void attack(); }       // "saldırabilmelisin" sözleşmesi (gövde yok)
interface Weapon { void use(); }          // "kullanılabilmelisin" sözleşmesi
class Warrior implements Fighter, Weapon {     // iki sözleşmeyi birden imzala (virgülle)
    public void attack() { System.out.println("Kılıçla saldırır"); }   // sözleşme 1'i yerine getir
    public void use() { System.out.println("Kılıcı kullanır"); }       // sözleşme 2'yi yerine getir
}
Warrior w = new Warrior();
w.attack();   // Kılıçla saldırır
w.use();      // Kılıcı kullanır
```

Benzetme: Interface = **yetenek sertifikası**. Bir kişi hem **ehliyet** hem **yüzme sertifikası** alabilir. Babası tek ama sertifikası çok olabilir. implements = "bu sertifikaların gerektirdiği her şeyi yapacağım" sözü.

122. İki interface aynı method'u getirirse

```java
interface A {
    default void hello() { System.out.println("A"); }   // default = gövdesi hazır method (Java 8+)
}
interface B {
    default void hello() { System.out.println("B"); }   // aynı isimde, farklı iş
}
class C implements A, B {
    @Override
    public void hello() {          // ZORUNLU: çakışmayı sen çöz
        A.super.hello();           // A'nınkini çalıştır
        B.super.hello();           // B'ninkini çalıştır
        System.out.println("C");   // üstüne kendininkini ekle
    }
}
new C().hello();   // A · B · C
```

Benzetme: İki öğretmen farklı ödev yöntemi verdi; **sen hangisini nasıl yapacağını açıkça söylemek zorundasın**. A.super.hello() = "A hocanın yöntemi".

123. Karma miras (hybrid)

```java
class C { public void disp() { System.out.println("C"); } }
class A extends C { public void disp() { System.out.println("A"); } }   // C'nin çocuğu
class B extends C { public void disp() { System.out.println("B"); } }   // C'nin diğer çocuğu
class D extends A { public void disp() { System.out.println("D"); } }   // A'nın çocuğu (C'nin torunu)
C[] objs = { new C(), new A(), new B(), new D() };
for (C o : objs) {
    o.disp();                                  // C · A · B · D → herkes kendi disp'ini çalıştırır
}
System.out.println(new D() instanceof C);      // true → torun da dedenin soyundandır
```

Şekil: Bir aile ağacı: C'nin iki çocuğu (A, B) + A'nın çocuğu (D). Kimse iki babadan almadığı için class'larla yazılabilir.

124. Şekil hiyerarşisi (Shape)

```java
class Shape {
    String name;
    Shape(String name) { this.name = name; }
    double area() { return 0; }                    // genel şeklin alanı bilinmez → 0
    void describe() {
        System.out.printf("%s alanı: %.2f%n", name, area());   // %s = yazı, %.2f = 2 ondalık, %n = alt satır
    }
}
class Circle extends Shape {
    double r;
    Circle(double r) { super("Daire"); this.r = r; }   // babaya ismi ver, yarıçapı kendin tut
    @Override double area() { return Math.PI * r * r; } // dairenin kendi formülü
}
class Rectangle extends Shape {
    double w, h;
    Rectangle(double w, double h) { super("Dikdörtgen"); this.w = w; this.h = h; }
    @Override double area() { return w * h; }          // dikdörtgenin kendi formülü
}
// Shape[] shapes = { new Circle(1), new Rectangle(3, 4) }; her birinde describe()
// Daire alanı: 3.14 · Dikdörtgen alanı: 12.00 · Toplam: 15.14
```

Benzetme: describe() **ortak sunucu** gibi: "...'nin alanı ..." der. Alanı hesaplamayı her şekil **kendisi** yapar. Yeni şekil eklemek için sadece yeni bir çocuk class yazarsın.

PDF 6 · Casting, final, aggregation

125. Upcasting, downcasting, instanceof

```java
class Animal { void eat() { System.out.println("eating"); } }
class Dog extends Animal { void bark() { System.out.println("woof"); } }
class Cat extends Animal { }

Animal a = new Dog();          // UPCASTING: köpeği "hayvan" etiketli kutuya koy (otomatik, güvenli)
a.eat();                       // eating
// a.bark();                   → HATA: kutunun etiketi "hayvan", Java havlamayı bilmiyor
if (a instanceof Dog) {        // instanceof = "bu gerçekten köpek mi?" diye sor
    Dog d = (Dog) a;           // DOWNCASTING: evet → "köpek" etiketli kutuya geri al (elle)
    d.bark();                  // woof
}
Animal b = new Cat();
if (b instanceof Dog dog) {    // Java 16+: hem sor hem dönüştür, tek adımda
    dog.bark();
} else {
    System.out.println("b bir Dog değil");   // b bir Dog değil
}
// Dog x = (Dog) b;            → ÇÖKER: ClassCastException (kediyi köpek yapamazsın)
```

Kavram	En basit anlamı
Upcasting	Çocuğu baba etiketiyle tutmak; **her zaman güvenli**
Downcasting	Baba etiketini tekrar çocuğa çevirmek; **elle (Dog) yazılır**
instanceof	"Bu nesne gerçekten o türden mi?" sorusu (true/false)
ClassCastException	Yanlış türe çevirmeye çalışınca program çöker

Benzetme: Köpeği **"evcil hayvan" yazan kafese** koyarsın (upcasting); kafes etiketi "evcil hayvan" olduğu için ona "havla" diyemezsin. Önce **"bu köpek mi?"** diye bakarsın (instanceof), sonra köpek kafesine alırsın (downcasting). Kediyi köpek kafesine koymaya çalışırsan **iş bozulur** (ClassCastException).

126. final: kilitleme

```
final class Utility {                          // final class = KİMSE miras alamaz
    static int twice(int x) { return 2 * x; }
}
// class MyUtil extends Utility { }            → HATA: cannot inherit from final Utility

class Account {
    final void audit() { }                     // final method = çocuklar DEĞİŞTİREMEZ
}
class Fake extends Account {
    // @Override void audit() { }              → HATA: overridden method is final
}
```

Benzetme: final class = **mühürlü tarif**: kimse kopyalayıp değiştiremez (Java'nın String, Integer, Math class'ları böyle). final method = **"bu kurala dokunma"**: güvenlik kontrolü gibi kritik işler kilitlenir.

127. Aggregation: işi başkasına yaptırmak

```
class Operation {
    int square(int n) { return n * n; }          // kare alma uzmanı
}
class Circle {
    Operation op;                                // Circle'ın bir Operation'ı VAR (HAS-A)
    Circle(Operation op) { this.op = op; }       // dışarıdan veriliyor
    double area(int radius) {
        return Math.PI * op.square(radius);      // kareyi kendisi almıyor, uzmana yaptırıyor
    }
}
Operation op = new Operation();                  // tek bir uzman
Circle c1 = new Circle(op);                      // c1 bu uzmanı kullanıyor
Circle c2 = new Circle(op);                      // c2 de AYNI uzmanı kullanıyor (paylaşım)
System.out.println(c1.area(5));                  // 78.53981633974483
System.out.println(c2.area(1));                  // 3.141592653589793
```

Benzetme: Evinde tamirci yok; **mahallenin ustasını** çağırırsın. Aynı usta başka evlere de gider (paylaşım). İşi kendin yapmak yerine ona **devredersin** (delegation).

128. Çalışan ve adresi

```
class Address {
    String city, country;
    Address(String city, String country) {
        this.city = city;
        this.country = country;
    }
}
class Emp {
    int id;
    String name;
    Address addr;                                // çalışanın bir adresi VAR
    Emp(int id, String name, Address addr) {
        this.id = id; this.name = name; this.addr = addr;
    }
    void display() {
        System.out.println(id + " " + name);
        System.out.println(" " + addr.city + ", " + addr.country);
    }
}
Address gms = new Address("Gümüşhane", "Türkiye");   // tek bir adres
Emp e1 = new Emp(111, "Varun", gms);                 // iki çalışan AYNI adresi kullanıyor
Emp e2 = new Emp(112, "Arun", gms);
e1.display();                                        // 111 Varun · Gümüşhane, Türkiye
e2.display();                                        // 112 Arun · Gümüşhane, Türkiye
gms.city = "Torul";                                  // adresi bir kez değiştir
e2.display();                                        // 112 Arun · Torul, Türkiye → değişti
```

Benzetme: İki çalışan **aynı lojmanda** oturuyor. Lojmanın adresi değişirse ikisinin de adresi değişir. Çalışan işten ayrılsa da **lojman yerinde durur** (aggregation = parça bağımsız yaşar).

129. Doğru ilişkiyi seçmek: araba motor değildir

```java
class Engine {
    void start() { System.out.println("Motor çalıştı"); }
}
// ✗ YANLIŞ: class Car extends Engine { }  → "araba bir motordur" saçma; çalışır ama yanlış model

//  ✓ DOĞRU: arabanın bir motoru VAR
class Car {
    private Engine engine = new Engine();    // motor arabanın İÇİNDE
    void start() { engine.start(); }         // araba çalış deyince motora yaptırır
}
new Car().start();                           // Motor çalıştı
```

Benzetme: "Araba bir motordur" cümlesi saçma → miras değil. **"Arabanın bir motoru var"** cümlesi doğru → HAS-A. Bu sayede motoru kolayca elektrikli motorla değiştirebilirsin.

	Inheritance (IS-A)	**Aggregation (HAS-A, zayıf)**	**Composition (HAS-A, güçlü)**
Cümle	"Köpek **bir** hayvandır"	"Çalışanın **bir** adresi var"	"Arabanın **bir** motoru var"
Kodda	extends	Dışarıdan verilir	İçeride new ile üretilir
Parça bütünsüz yaşar mı?	—	**Evet**	**Hayır**
Paylaşılır mı?	—	Evet	Hayır
Benzetme	Baba–oğul	Öğretmen–okul	Kalp–insan